MANUAL PASO A PASO CON TDD

Punto de Venta

Flutter + SQLite local · Dominio puro · Transacciones

Desarrollo Móvil Integral · UTNG

Prof. Anastacio Rodríguez García

Antes de empezar: ¿qué vamos a construir?

Imagina la nevería de la plaza de Dolores Hidalgo un domingo por la tarde: la fila llega a la puerta, el cajero cobra de memoria, alguien pide "tres de limón y una de tequila", paga con un billete de $200 y todos esperan el cambio. Si el cajero se equivoca de $5 en cada venta, al final de la semana faltan cientos de pesos. Hoy construiremos la app que evita esos errores.

La app en una frase

Un punto de venta que guarda productos y ventas en una base de datos SQLite dentro del teléfono (funciona sin internet), calcula totales, IVA y cambio sin errores, descuenta el inventario automáticamente y hace el corte del día.

Lo que aprenderás

- Por qué el dinero NUNCA se guarda en double y cómo usar centavos enteros.
- Escribir primero la prueba y después el código (TDD) en 3 capas: dominio, datos y presentación.
- Qué es una transacción de base de datos y por qué evita que se pierda inventario o dinero.
- Probar SQLite en tu computadora, sin emulador, usando una base en memoria.
- Conectar todo con un ViewModel y una pantalla Flutter.

Mapa del proyecto

```
punto_venta_dolores/
  lib/
    dominio/        <- reglas del negocio (Dart puro, sin Flutter)
      dinero.dart  producto.dart  carrito.dart  venta.dart  excepciones.dart
    datos/          <- SQLite
      base_datos.dart  repositorio_productos.dart  repositorio_ventas.dart
    presentacion/   <- lo que ve el usuario
      caja_viewmodel.dart  pantalla_caja.dart
    main.dart       <- aquí se conecta todo (composition root)
  test/
    dominio_test.dart   datos_test.dart
```

El ciclo que repetiremos en cada ronda

🔴 ROJO: escribe la prueba (debe FALLAR)
Escribes una prueba de lo que el programa DEBERÍA hacer. Como aún no existe el código, la prueba falla. Eso es bueno: demuestra que la prueba sí detecta errores.

🟢 VERDE: el código mínimo para pasar
Escribes el código más simple posible para que la prueba pase. Sin adornos.

🔵 REFACTOR: ordena sin romper nada
Con la prueba en verde, mejoras nombres y estructura. Si algo se rompe, la prueba te avisa al instante.

💡 Dato curioso / ejemplo
Es como un cajero que cuenta el cambio dos veces: la prueba es la segunda cuenta. Si las dos coinciden, el cliente sale feliz.

Ronda 0 · Preparar el taller

Paso 0.1 · Crear el proyecto

Abre una terminal y ejecuta estos comandos. Así tendrás un proyecto Flutter limpio.

```
$ flutter create punto_venta_dolores
$ cd punto_venta_dolores
```

Paso 0.2 · Instalar las dependencias

Reemplaza el contenido de pubspec.yaml por el siguiente. Fíjate en dos detalles: sqflite es el SQLite para el teléfono, y sqflite_common_ffi es su versión para correr pruebas en tu computadora.

📄 **pubspec.yaml**

```
name: punto_venta_dolores
description: Punto de venta con SQLite local para negocios de Dolores Hidalgo.
publish_to: none
version: 1.0.0

environment:
  sdk: ">=3.0.0 <4.0.0"

dependencies:
  flutter:
    sdk: flutter
  sqflite: ^2.3.0
  path: ^1.8.3

dev_dependencies:
  flutter_test:
    sdk: flutter
  sqflite_common_ffi: ^2.3.0
  flutter_lints: ^3.0.0

flutter:
  uses-material-design: true
```

```
$ flutter pub get
```

Paso 0.3 · Crear las carpetas

```
$ mkdir lib/dominio lib/datos lib/presentacion
```

Paso 0.4 · Verificar que las pruebas corren

Borra test/widget_test.dart (viene de ejemplo y ya no sirve) y ejecuta:

```
$ flutter test
```

> 💡 **Dato curioso / ejemplo**
> Si ves "No tests ran" está bien: el taller está limpio. Empezamos con tarea cero: tener el semáforo listo antes de manejar.

Ronda 1 · El dinero: la regla de oro

El problema que casi nadie ve

Abre DartPad y escribe print(0.1 + 0.2). Verás 0.30000000000000004. Las computadoras guardan los decimales en binario y algunos números (como 0.1) no caben exactos, igual que 1/3 no cabe exacto en decimal (0.3333...). Si cobras 1,000 ventas con ese error diminuto, las cuentas del corte NO cuadran.

> 💡 **Dato curioso / ejemplo**
> En 1991, un misil Patriot falló por acumular un error de redondeo de solo 0.000000095 segundos por décima de segundo. Pequeños errores se acumulan. En un punto de venta, ese error son pesos desaparecidos.

La solución: centavos enteros

Guardamos $25.00 como 2500 (centavos). Los enteros no tienen error de redondeo. Solo convertimos a "$25.00" cuando mostramos en pantalla.

Paso 1.1 · Primero las excepciones

Son los errores del negocio con nombre propio. Tener "ExistenciasInsuficientesException" es mucho más claro que un genérico "Exception". Las usaremos desde la ronda 3.

📄 **lib/dominio/excepciones.dart**

```
/// Errores de negocio. Dart puro: se prueban sin Flutter ni base de datos.
class CodigoDuplicadoException implements Exception {
  final String codigo;
  CodigoDuplicadoException(this.codigo);
  @override
  String toString() => 'Ya existe un producto con el código $codigo';
}

class ExistenciasInsuficientesException implements Exception {
  final String producto;
  final int disponibles;
  ExistenciasInsuficientesException(this.producto, this.disponibles);
  @override
  String toString() => 'Solo quedan $disponibles de $producto';
}

class PagoInsuficienteException implements Exception {
  final int faltanCentavos;
  PagoInsuficienteException(this.faltanCentavos);
  @override
  String toString() => 'Faltan $faltanCentavos centavos para cubrir la cuenta';
}

class CarritoVacioException implements Exception {
  @override
  String toString() => 'No hay productos en el carrito';
}
```

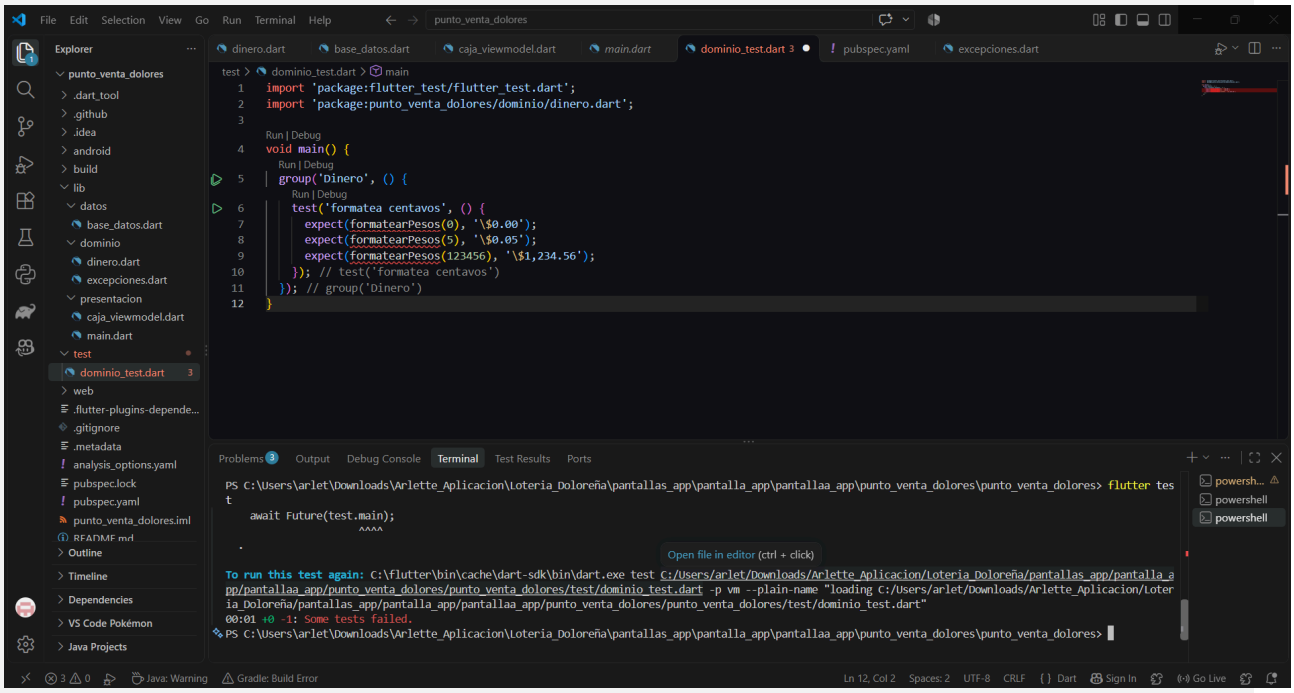
Paso 1.2 · Prueba ROJA: formatear pesos

📄 test/dominio_test.dart

```
import 'package:flutter_test/flutter_test.dart';
import 'package:punto_venta_dolores/dominio/dinero.dart';

void main() {
  group('Dinero', () {
    test('formatea centavos', () {
      expect(formatearPesos(0), '\$0.00');
      expect(formatearPesos(5), '\$0.05');
      expect(formatearPesos(123456), '\$1,234.56');
    });
  });
}
```

```
$ flutter test
```



> **🔴 ROJO: escribe la prueba (debe FALLAR)**
> Verás un error rojo: "Target of URI doesn't exist". Perfecto, el archivo dinero.dart aún no existe. Esa falla es nuestra brújula.

Paso 1.3 · Verde: implementar

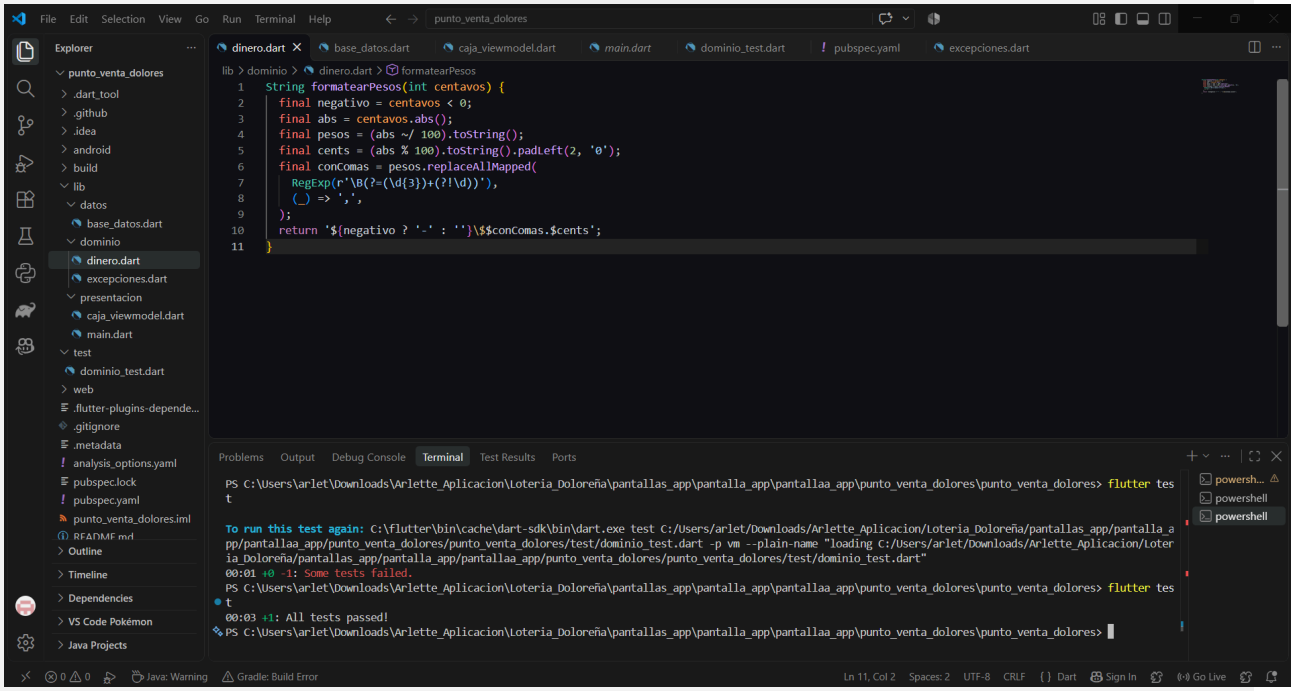
Creamos dinero.dart. Primero haremos solo formatearPesos; pesosACentavos lo agregaremos en la siguiente prueba.

📄 lib/dominio/dinero.dart (versión 1)

```
String formatearPesos(int centavos) {
  final negativo = centavos < 0;
  final abs = centavos.abs();
  final pesos = (abs ~/ 100).toString();
  final cents = (abs % 100).toString().padLeft(2, '0');
  final conComas = pesos.replaceAllMapped(
    RegExp(r'\B(?=(\d{3})+(?!\d))'),
    (_) => ',',
```

```
  );
  return '${negativo ? '-' : ''}\$$conComas.$cents';
}
```

```
$ flutter test
```



> 🟢 **VERDE: el código mínimo para pasar**
> Los tres casos pasan. Nota cómo cada línea responde a un caso: padLeft hace que 5 centavos sea "05", y la expresión regular inserta las comas de miles.

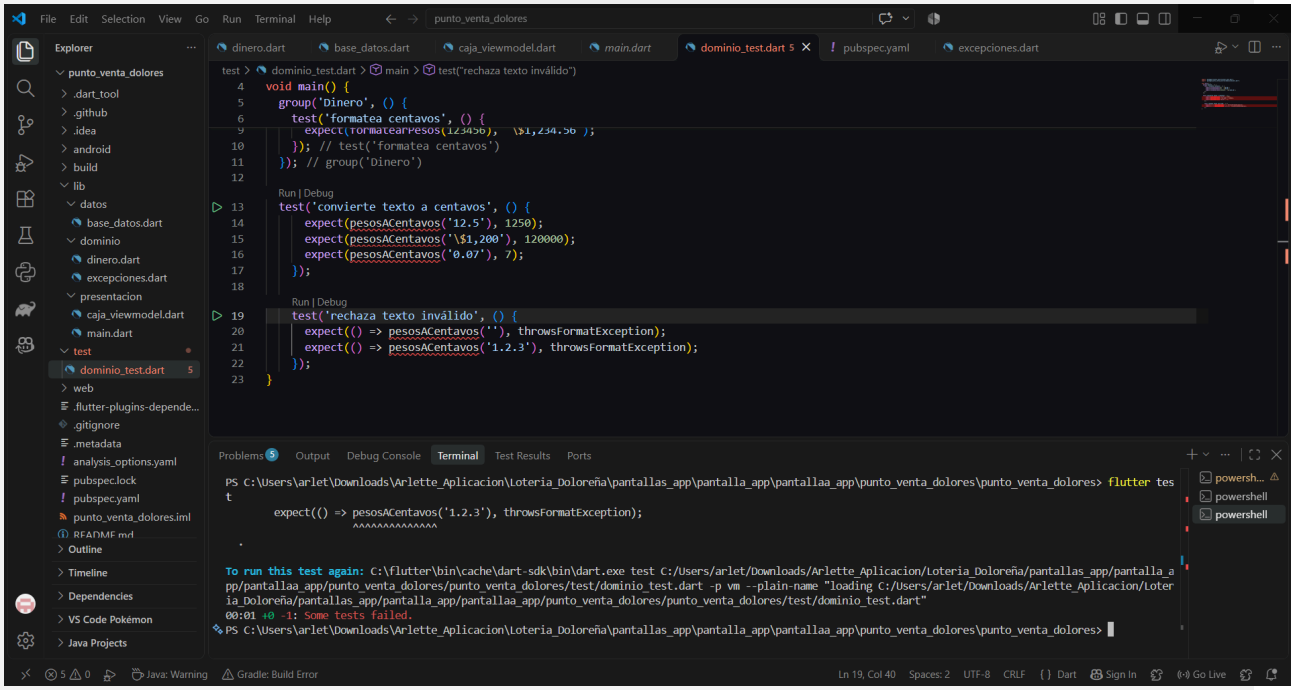
> 💡 **Dato curioso / ejemplo**
> El truco ~/ es la división entera de Dart: 123456 ~/ 100 = 1234 (sin decimales). Y % da el residuo: 123456 % 100 = 56 centavos. Es como repartir pizza: cuántas rebanadas completas y cuántas sobran.

Paso 1.4 · Segunda prueba: texto a centavos

El cajero escribirá "12.5" en el teclado. Debemos convertirlo a 1250 centavos sin usar double.

> 🔴 **ROJO: escribe la prueba (debe FALLAR)**
> Agrega dentro del group('Dinero'):

```
    test('convierte texto a centavos', () {
      expect(pesosACentavos('12.5'), 1250);
      expect(pesosACentavos('\$1,200'), 120000);
      expect(pesosACentavos('0.07'), 7);
    });
    test('rechaza texto inválido', () {
      expect(() => pesosACentavos(''), throwsFormatException);
      expect(() => pesosACentavos('1.2.3'), throwsFormatException);
    });
```



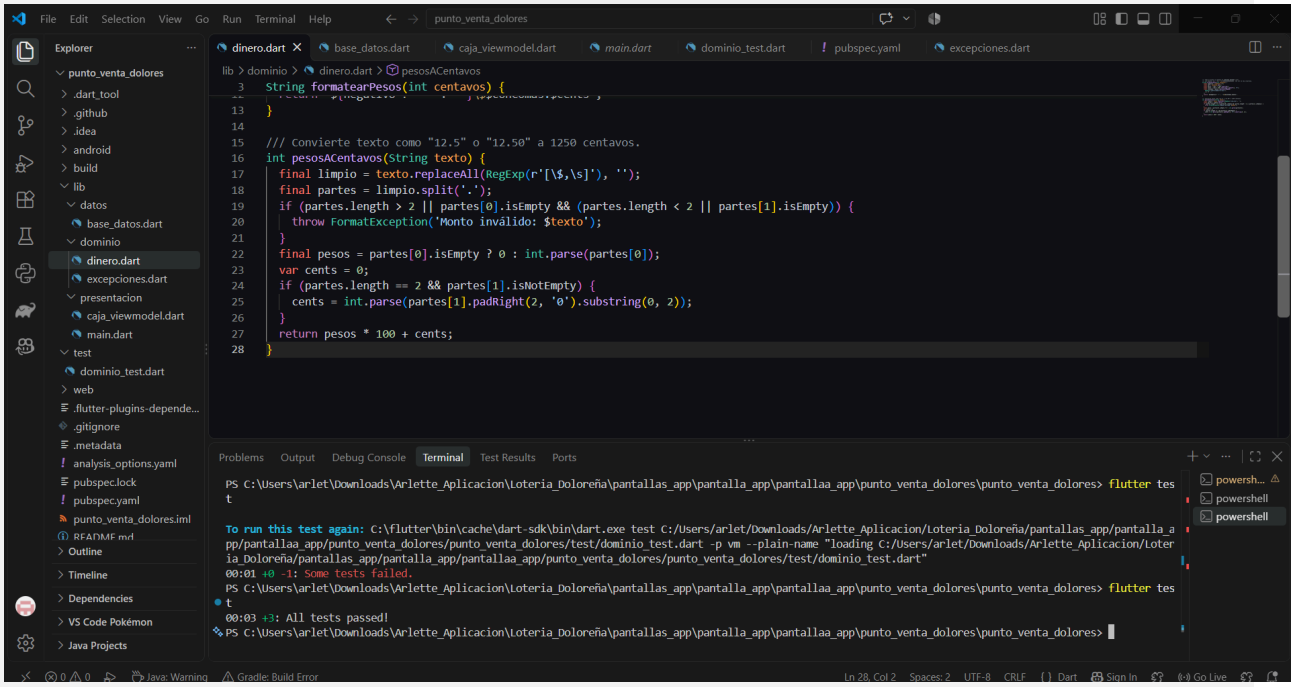
🟢 VERDE: el código mínimo para pasar

Reemplaza dinero.dart por la versión completa:

📄 lib/dominio/dinero.dart (versión final)

```
/// Todo el dinero se maneja en CENTAVOS ENTEROS (int).
/// Con double: 0.1 + 0.2 = 0.30000000000000004. Con int no hay sorpresas.
String formatearPesos(int centavos) {
  final negativo = centavos < 0;
  final abs = centavos.abs();
  final pesos = (abs ~/ 100).toString();
  final cents = (abs % 100).toString().padLeft(2, '0');
  final conComas = pesos.replaceAllMapped(
    RegExp(r'\B(?=(\d{3})+(?!\d))'),
    (_) => ',',
  );
  return '${negativo ? '-' : ''}\$$conComas.$cents';
}

/// Convierte texto como "12.5" o "12.50" a 1250 centavos.
int pesosACentavos(String texto) {
  final limpio = texto.replaceAll(RegExp(r'[\$,\s]'), '');
  final partes = limpio.split('.');
  if (partes.length > 2 || partes[0].isEmpty && (partes.length < 2 || partes[1].isEmpty)) {
    throw FormatException('Monto inválido: $texto');
  }
  final pesos = partes[0].isEmpty ? 0 : int.parse(partes[0]);
  var cents = 0;
  if (partes.length == 2 && partes[1].isNotEmpty) {
    cents = int.parse(partes[1].padRight(2, '0').substring(0, 2));
  }
  return pesos * 100 + cents;
}
```



> ⚠️ **Cuidado**
> "12.5" son 12 pesos con 50 centavos, no 12 pesos con 5. Por eso usamos padRight(2, '0'): "5" se vuelve "50". Esta prueba atrapa justo ese error clásico de principiante.

> 🏆 **RETO PARA TI**
> Agrega una prueba para pesosACentavos('.5') (debería dar 50) y otra para formatearPesos(-250) (debería dar "-$2.50"). ¿Pasan sin cambiar código?

🏅 **INSIGNIA DESBLOQUEADA: Guardián de los centavos**

Ronda 2 · El Producto

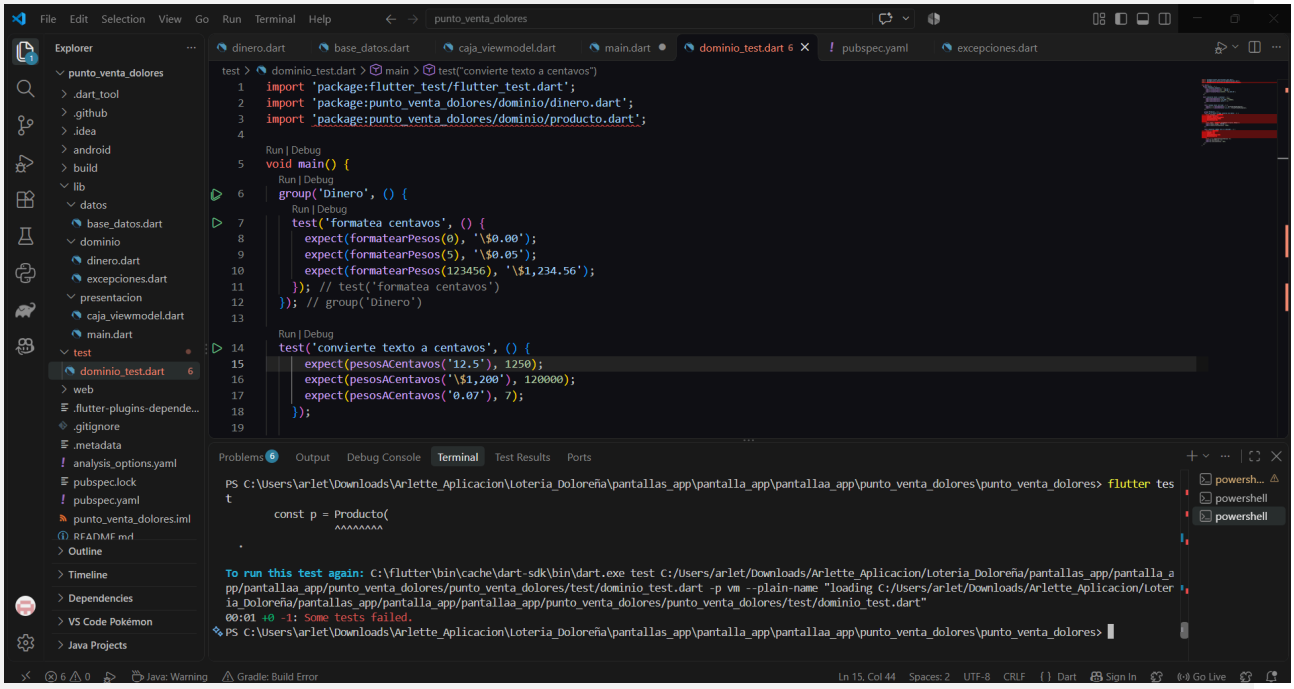
Un producto tiene código, nombre, precio (en centavos), existencias y, cuando viene de la base de datos, un id. Además necesita viajar a la base como un Map y regresar. Probamos ese viaje de ida y vuelta.

Paso 2.1 · Prueba ROJA

📄 **test/dominio_test.dart (agrega arriba, junto a los imports)**

```
import 'package:punto_venta_dolores/dominio/producto.dart';

// dentro de main(), nuevo group:
  group('Producto', () {
    test('ida y vuelta a mapa conserva los datos', () {
      const original = Producto(
        codigo: 'NV-001', nombre: 'Nieve de limón',
        precioCentavos: 2500, existencias: 10,
      );
      final copia = Producto.desdeMapa(original.aMapa());
      expect(copia.codigo, 'NV-001');
      expect(copia.precioCentavos, 2500);
    });
    test('copiarCon cambia solo lo indicado', () {
      const p = Producto(codigo: 'A', nombre: 'X', precioCentavos: 100, existencias: 5);
      final q = p.copiarCon(existencias: 4);
      expect(q.existencias, 4);
      expect(q.precioCentavos, 100);
    });
  });
```



Paso 2.2 · VERDE

📄 **lib/dominio/producto.dart**

```
class Producto {
  final int? id;
```

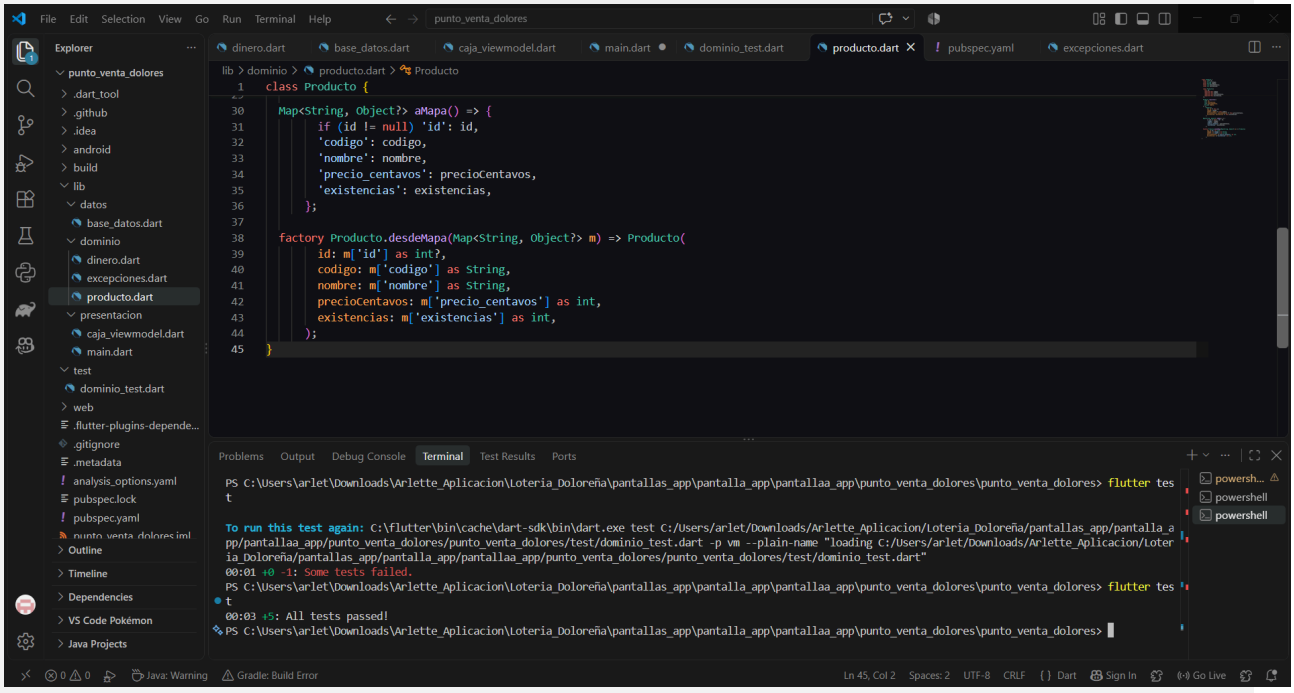
```dart
  final String codigo;
  final String nombre;
  final int precioCentavos;
  final int existencias;

  const Producto({
    this.id,
    required this.codigo,
    required this.nombre,
    required this.precioCentavos,
    required this.existencias,
  });

  Producto copiarCon({int? id, int? existencias, int? precioCentavos, String? nombre}) =>
      Producto(
        id: id ?? this.id,
        codigo: codigo,
        nombre: nombre ?? this.nombre,
        precioCentavos: precioCentavos ?? this.precioCentavos,
        existencias: existencias ?? this.existencias,
      );

  Map<String, Object?> aMapa() => {
        if (id != null) 'id': id,
        'codigo': codigo,
        'nombre': nombre,
        'precio_centavos': precioCentavos,
        'existencias': existencias,
      };

  factory Producto.desdeMapa(Map<String, Object?> m) => Producto(
        id: m['id'] as int?,
        codigo: m['codigo'] as String,
        nombre: m['nombre'] as String,
        precioCentavos: m['precio_centavos'] as int,
        existencias: m['existencias'] as int,
      );
}
```



💡 **Dato curioso / ejemplo**

Los objetos son inmutables (todo final). Si el precio cambia creas una copia nueva con copiarCon. Es como una credencial: no le borras el nombre, imprimes una nueva. Así nadie modifica un producto "por accidente" desde otra parte del código.

🔵 **REFACTOR: ordena sin romper nada**

Observa cómo aMapa usa nombres con guion bajo (precio_centavos) porque así se llaman las columnas SQL, mientras que el código Dart usa camelCase. El mapa es el "traductor" entre los dos idiomas.

🏅 **INSIGNIA DESBLOQUEADA: Arquitecto de datos**

Ronda 3 · El Carrito (el corazón del negocio)

Esta es la ronda más importante: aquí viven las reglas. Haremos cuatro mini-ciclos. Prepara un ayudante al inicio del archivo de pruebas:

📄 **test/dominio_test.dart (antes de main)**

```
import 'package:punto_venta_dolores/dominio/carrito.dart';
import 'package:punto_venta_dolores/dominio/excepciones.dart';

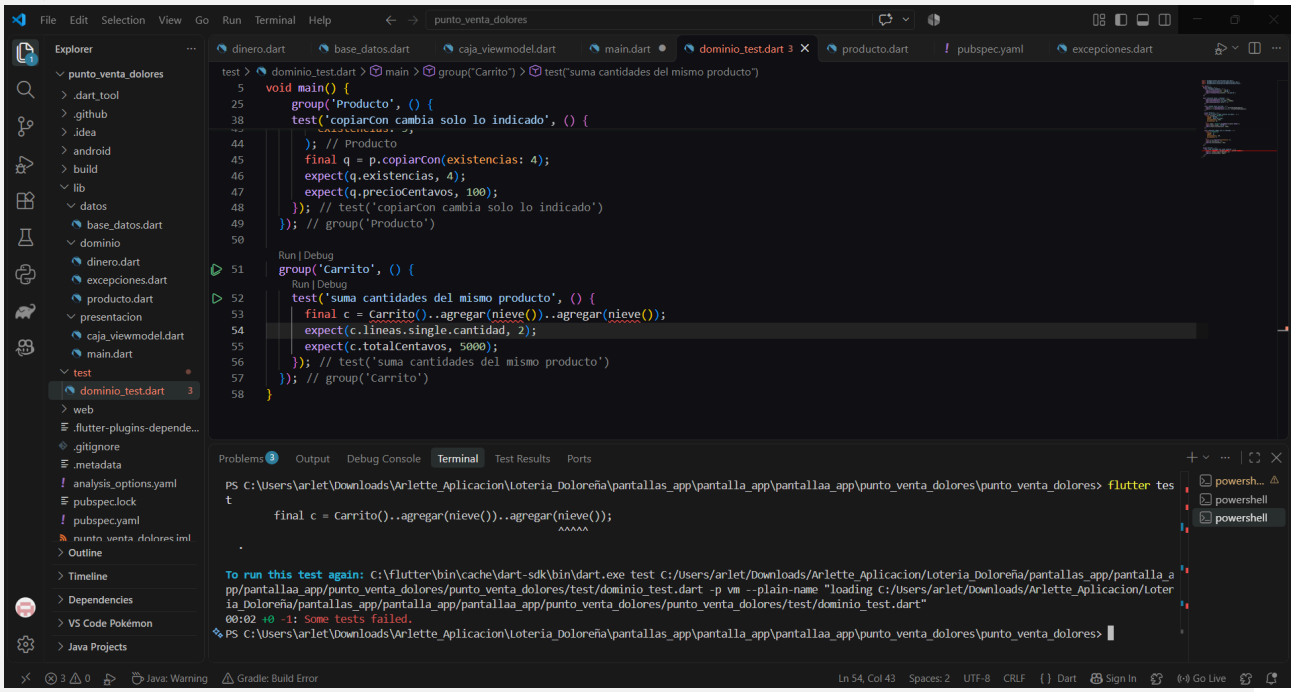
Producto nieve([int existencias = 10]) => Producto(
    codigo: 'NV-001', nombre: 'Nieve',
    precioCentavos: 2500, existencias: existencias);
```

Ciclo 3.1 · Sumar cantidades

🔴 **ROJO: escribe la prueba (debe FALLAR)**

Si el cliente pide dos nieves, NO debe haber dos renglones: un renglón con cantidad 2.

```
group('Carrito', () {
  test('suma cantidades del mismo producto', () {
    final c = Carrito()..agregar(nieve())..agregar(nieve());
    expect(c.lineas.single.cantidad, 2);
    expect(c.totalCentavos, 5000);
  });
});
```



🟢 **VERDE: el código mínimo para pasar**

Versión mínima de carrito.dart (aún sin validaciones ni IVA real):

📄 **lib/dominio/carrito.dart (versión 1)**

```
import 'producto.dart';

class LineaCarrito {
```

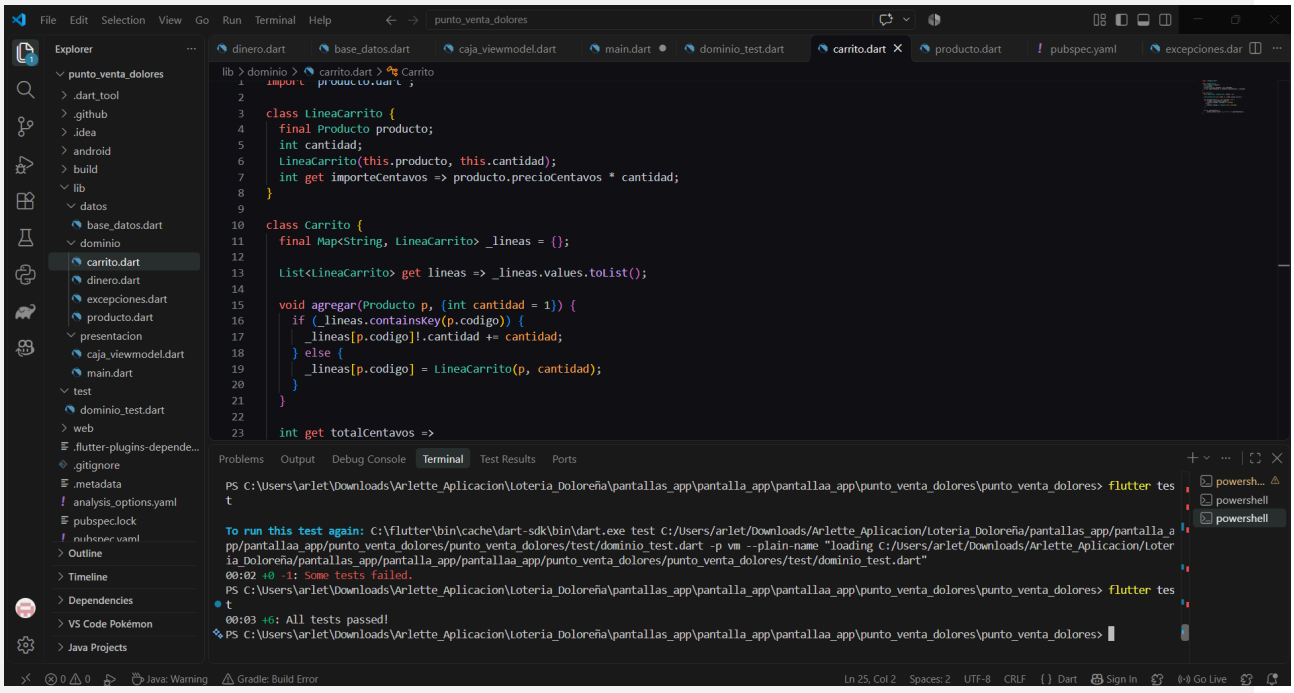
```
  final Producto producto;
  int cantidad;
  LineaCarrito(this.producto, this.cantidad);
  int get importeCentavos => producto.precioCentavos * cantidad;
}

class Carrito {
  final Map<String, LineaCarrito> _lineas = {};

  List<LineaCarrito> get lineas => _lineas.values.toList();

  void agregar(Producto p, {int cantidad = 1}) {
    if (_lineas.containsKey(p.codigo)) {
      _lineas[p.codigo]!.cantidad += cantidad;
    } else {
      _lineas[p.codigo] = LineaCarrito(p, cantidad);
    }
  }

  int get totalCentavos =>
      _lineas.values.fold(0, (s, l) => s + l.importeCentavos);
}
```



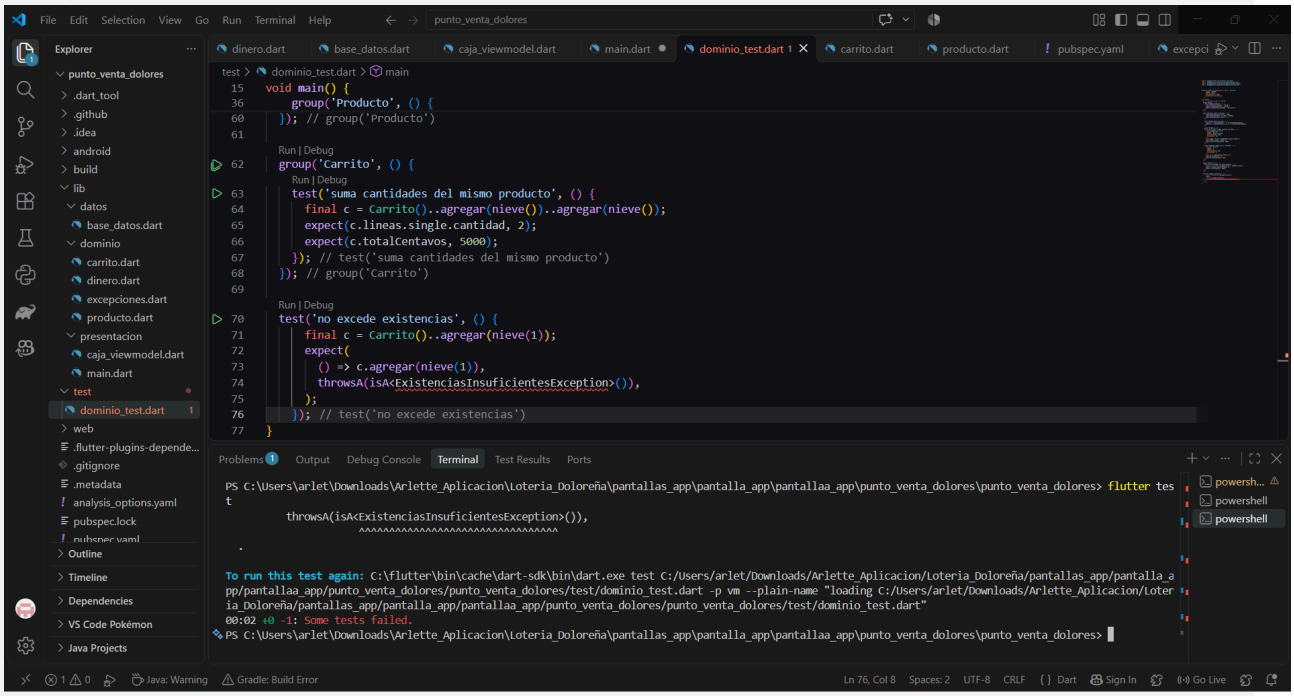
> 💡 **Dato curioso / ejemplo**
> Usamos un Map con el código como llave. Es como la libreta de la tiendita: cada producto tiene UNA línea, y si repite, solo subes el número. Buscar por llave es instantáneo, aunque tengas 10,000 productos.

Ciclo 3.2 · No vender lo que no hay

> 🔴 **ROJO: escribe la prueba (debe FALLAR)**
> Solo queda 1 nieve; el cliente intenta llevarse 2.

```
    test('no excede existencias', () {
      final c = Carrito()..agregar(nieve(1));
      expect(() => c.agregar(nieve(1)),
          throwsA(isA<ExistenciasInsuficientesException>()));
```

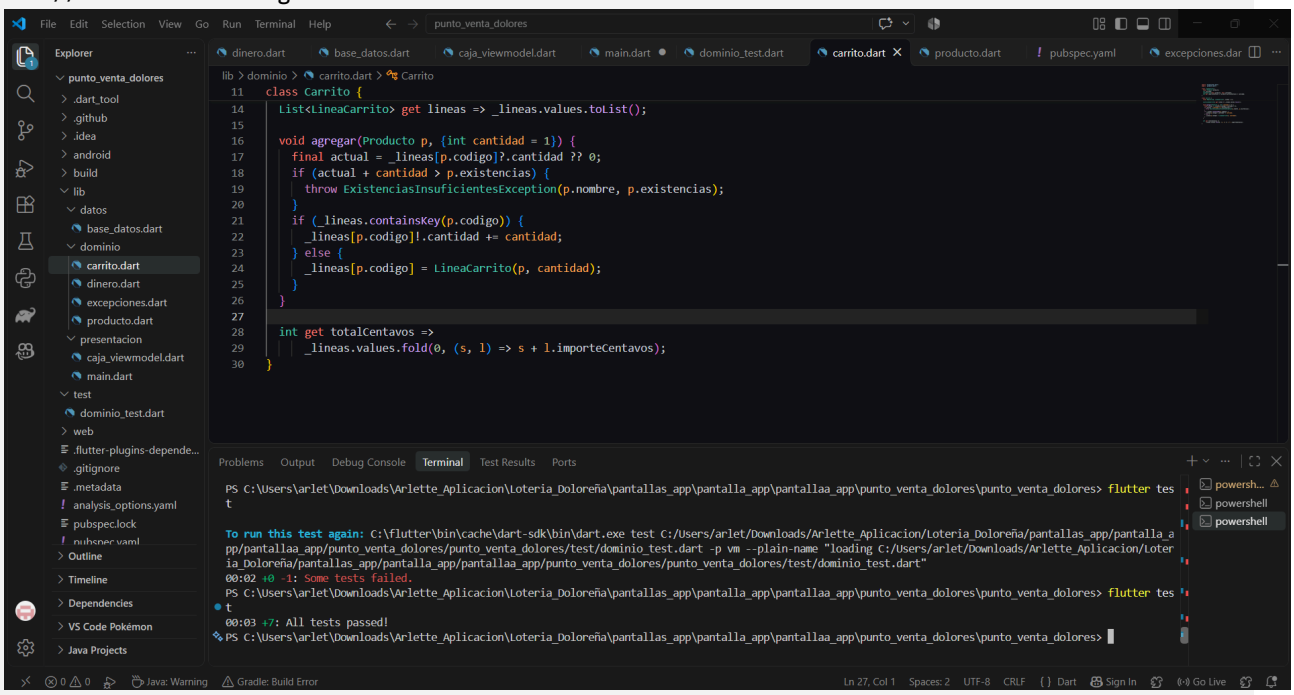
```
});
```



🟢 VERDE: el código mínimo para pasar

Dentro de agregar, antes de modificar nada, valida:

```
void agregar(Producto p, {int cantidad = 1}) {
  final actual = _lineas[p.codigo]?.cantidad ?? 0;
  if (actual + cantidad > p.existencias) {
    throw ExistenciasInsuficientesException(p.nombre, p.existencias);
  }
  // ... el resto igual
```



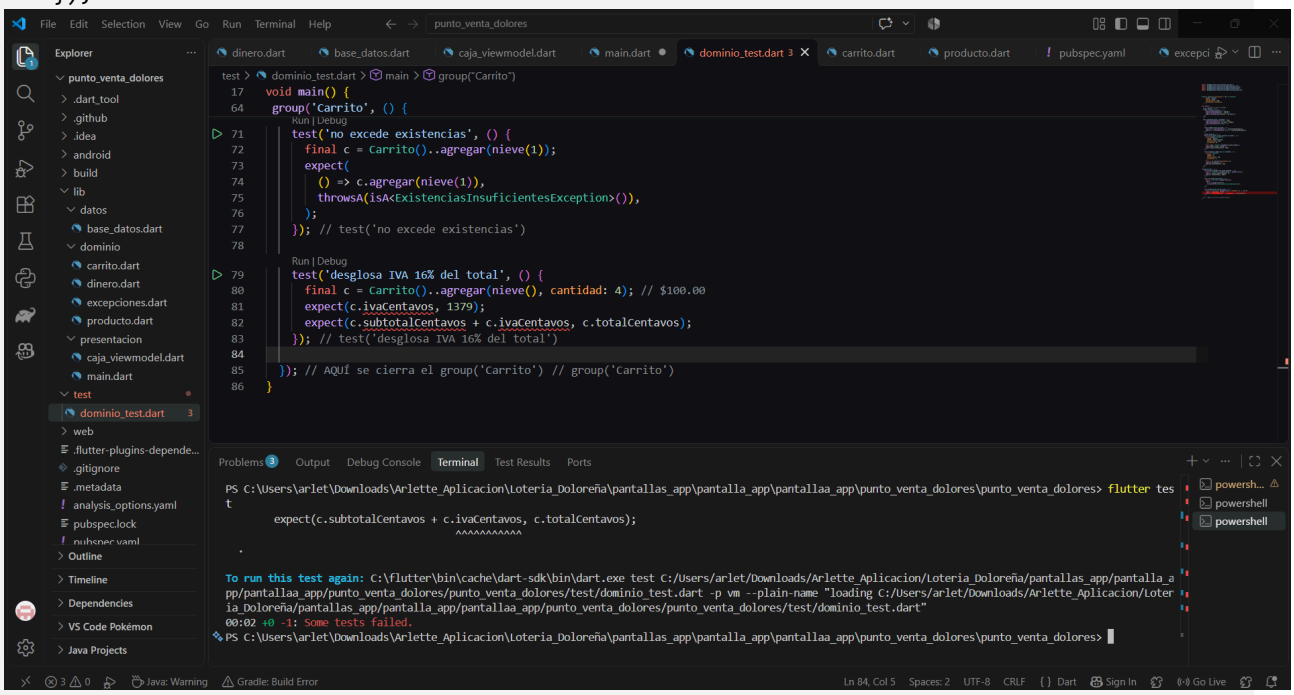
> ⚠️ **Cuidado**
> Valida ANTES de cambiar el estado. Si primero sumas y luego validas, el carrito queda con datos inválidos aunque lances el error.

Ciclo 3.3 · IVA incluido, con enteros

En México los precios al público ya incluyen IVA (16%). Para el ticket debemos desglosar: total = subtotal + IVA. La fórmula: iva = total × 16 / 116. Con $100.00 (10,000 centavos): 10000 × 16 / 116 = 1379.31... ⟶ 1379 centavos ($13.79).

> 🔴 **ROJO: escribe la prueba (debe FALLAR)**
> Dos cosas deben cumplirse: el valor exacto, y que subtotal + IVA siempre dé el total (cero centavos perdidos).

```
  test('desglosa IVA 16% del total', () {
    final c = Carrito()..agregar(nieve(), cantidad: 4); // \$100.00
    expect(c.ivaCentavos, 1379);
    expect(c.subtotalCentavos + c.ivaCentavos, c.totalCentavos);
  });
```



> 🟢 **VERDE: el código mínimo para pasar**
> Versión final de carrito.dart completa (con tasa configurable, quitar, vaciar y cambio):

📄 **lib/dominio/carrito.dart (versión final)**

```
import 'excepciones.dart';
import 'producto.dart';

class LineaCarrito {
  final Producto producto;
  int cantidad;
  LineaCarrito(this.producto, this.cantidad);
  int get importeCentavos => producto.precioCentavos * cantidad;
}
```

```
class Carrito {
  /// IVA en porcentaje entero (16 = 16%).
  final int tasaIva;
  final Map<String, LineaCarrito> _lineas = {};

  Carrito({this.tasaIva = 16});

  List<LineaCarrito> get lineas => _lineas.values.toList();
  bool get estaVacio => _lineas.isEmpty;

  void agregar(Producto p, {int cantidad = 1}) {
    final actual = _lineas[p.codigo]?.cantidad ?? 0;
    if (actual + cantidad > p.existencias) {
      throw ExistenciasInsuficientesException(p.nombre, p.existencias);
    }
    if (_lineas.containsKey(p.codigo)) {
      _lineas[p.codigo]!.cantidad += cantidad;
    } else {
      _lineas[p.codigo] = LineaCarrito(p, cantidad);
    }
  }

  void quitar(String codigo) => _lineas.remove(codigo);

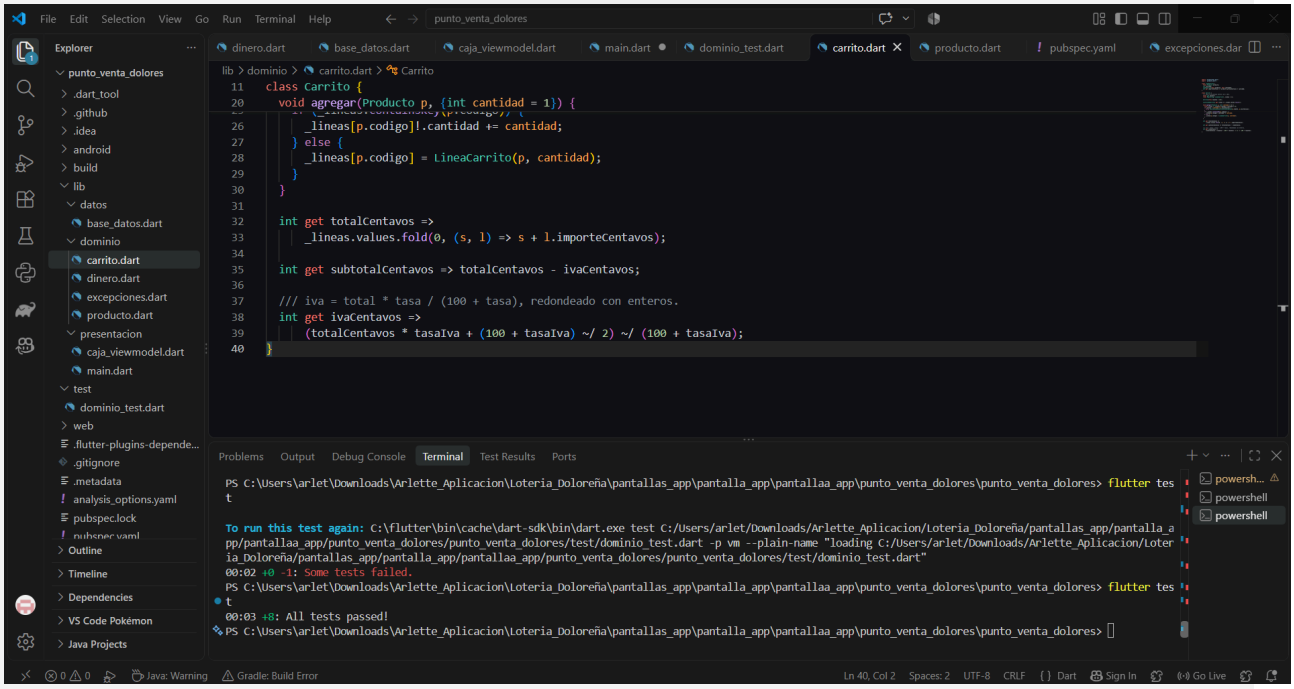
  void vaciar() => _lineas.clear();

  /// Los precios YA incluyen IVA: se desglosa el impuesto del total.
  int get totalCentavos =>
      _lineas.values.fold(0, (s, l) => s + l.importeCentavos);

  int get subtotalCentavos => totalCentavos - ivaCentavos;

  /// iva = total * tasa / (100 + tasa), redondeado con enteros.
  int get ivaCentavos =>
      (totalCentavos * tasaIva + (100 + tasaIva) ~/ 2) ~/ (100 + tasaIva);

  int cambio(int pagoCentavos) {
    if (estaVacio) throw CarritoVacioException();
    if (pagoCentavos < totalCentavos) {
      throw PagoInsuficienteException(totalCentavos - pagoCentavos);
    }
    return pagoCentavos - totalCentavos;
  }
}
```

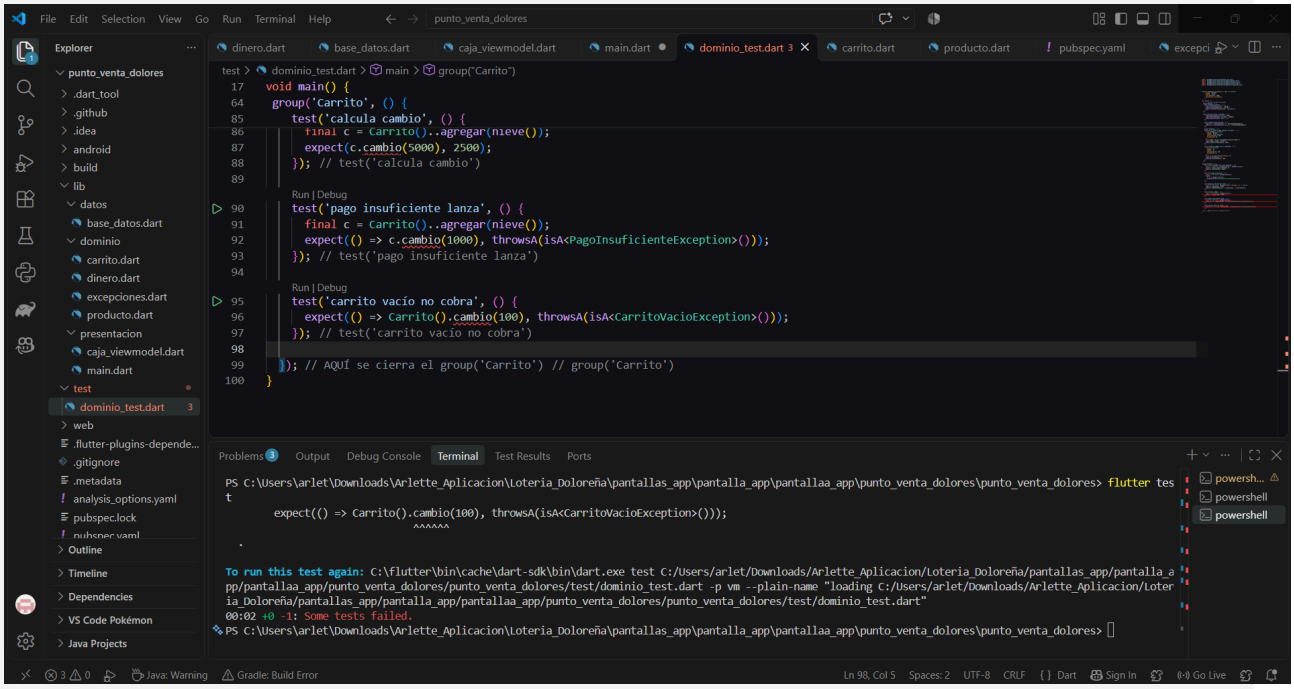


> 💡 **Dato curioso / ejemplo**
> Redondear con enteros: sumamos la mitad del divisor ((100+tasa) ~/ 2) antes de dividir. Es el mismo truco de redondear 7.6 a 8: sumas 0.5 y quitas decimales. Y calculamos el subtotal como total - iva, para que NUNCA falte ni sobre un centavo.

Ciclo 3.4 · Cambio, pago corto y carrito vacío

> 🔴 **ROJO: escribe la prueba (debe FALLAR)**
> Tres situaciones reales de caja:

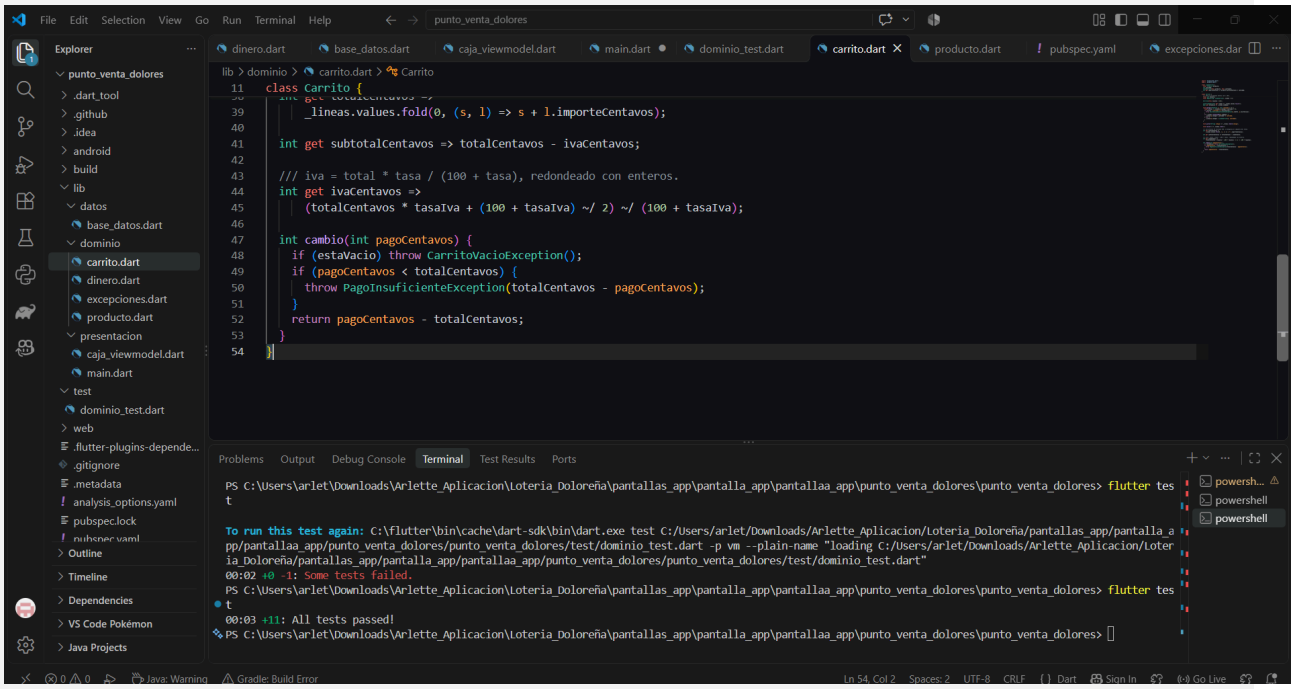
```
test('calcula cambio', () {
  final c = Carrito()..agregar(nieve());
  expect(c.cambio(5000), 2500);
});
test('pago insuficiente lanza', () {
  final c = Carrito()..agregar(nieve());
  expect(() => c.cambio(1000), throwsA(isA<PagoInsuficienteException>()));
});
test('carrito vacío no cobra', () {
  expect(() => Carrito().cambio(100), throwsA(isA<CarritoVacioException>()));
});
```



🟢 VERDE: el código mínimo para pasar

El método cambio() ya está en la versión final de arriba. Ejecuta flutter test: las 11 pruebas del dominio deben pasar.

```
$ flutter test test/dominio_test.dart
```



🔵 REFACTOR: ordena sin romper nada

Fíjate que el dominio no importa Flutter ni SQLite. Por eso esas 11 pruebas corren en milisegundos. Esa es la gran ventaja de separar capas.

Paso 3.5 · Las clases de Venta

Una venta guardada es una "foto" de lo que pasó: total, IVA, pago y renglones. Copia el archivo tal cual; son clases simples de datos.

📄 **lib/dominio/venta.dart**

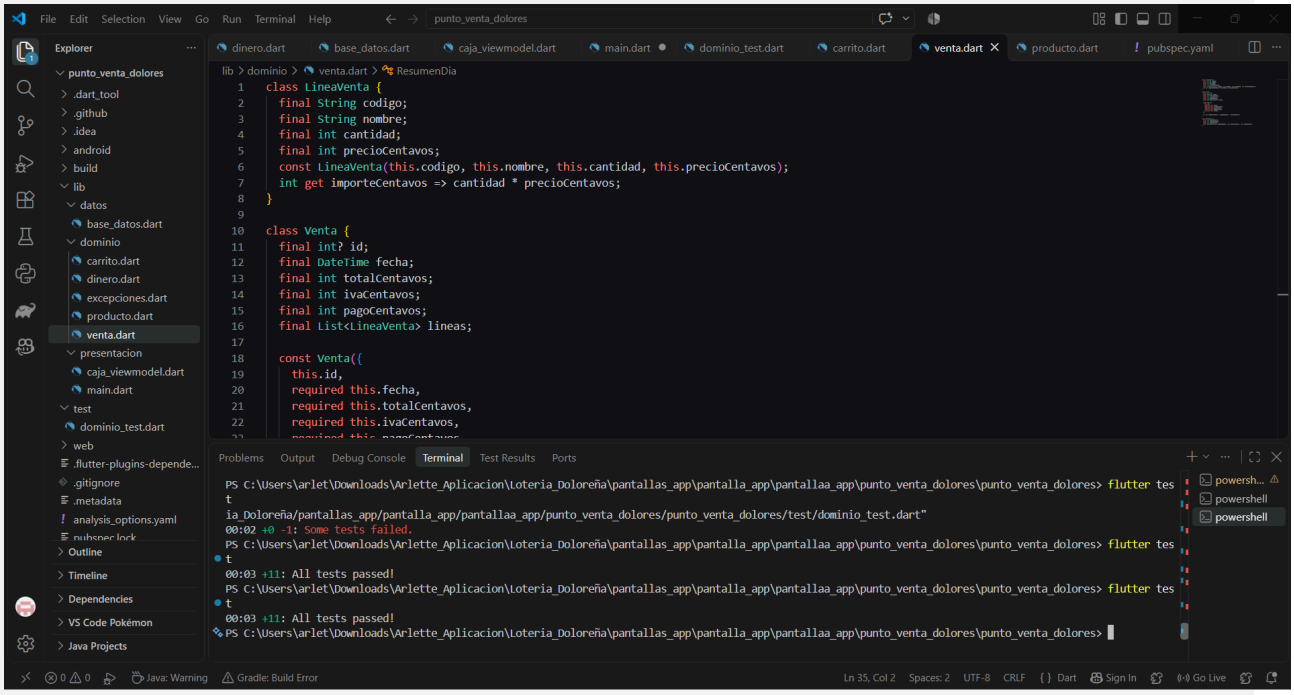
```
class LineaVenta {
  final String codigo;
  final String nombre;
  final int cantidad;
  final int precioCentavos;
  const LineaVenta(this.codigo, this.nombre, this.cantidad, this.precioCentavos);
  int get importeCentavos => cantidad * precioCentavos;
}

class Venta {
  final int? id;
  final DateTime fecha;
  final int totalCentavos;
  final int ivaCentavos;
  final int pagoCentavos;
  final List<LineaVenta> lineas;

  const Venta({
    this.id,
    required this.fecha,
    required this.totalCentavos,
    required this.ivaCentavos,
    required this.pagoCentavos,
    required this.lineas,
  });

  int get cambioCentavos => pagoCentavos - totalCentavos;
}

class ResumenDia {
  final int numeroVentas;
  final int totalCentavos;
  final int ivaCentavos;
  const ResumenDia(this.numeroVentas, this.totalCentavos, this.ivaCentavos);
}
```



💡 Dato curioso / ejemplo

¿Por qué LineaVenta guarda el nombre y precio y no solo referencia al producto? Porque mañana el precio cambia. Un ticket de hoy debe mostrar lo que se cobró hoy, igual que el recibo de papel que no cambia aunque la tienda suba precios.

🏆 RETO PARA TI

Agrega descuento: un método aplicarDescuento(int porcentaje) en Carrito. Escribe la prueba primero (10% sobre $100.00 → total $90.00). ¿Qué pasa con el IVA?

🏅 INSIGNIA DESBLOQUEADA: Maestro del carrito

Ronda 4 · Base de datos local (SQLite)

¿Qué es SQLite y por qué aquí?

SQLite es una base de datos completa que vive en UN archivo dentro del teléfono. No necesita servidor ni internet. Es la misma tecnología que usan WhatsApp, Chrome y tu teléfono para guardar datos. Perfecto para una tiendita en el mercado donde a veces falla la señal.

> 💡 **Dato curioso / ejemplo**
> SQLite es probablemente el software más desplegado del mundo: está en miles de millones de dispositivos. Cada vez que abres una app de mensajes o fotos, casi seguro hay un archivo SQLite detrás.

Paso 4.1 · Las tablas

Tres tablas: productos, ventas y lineas_venta. Una venta tiene muchas líneas (relación uno a muchos), unidas por venta_id. Nota los UNIQUE y NOT NULL: son reglas que la base hace cumplir aunque el código falle.

📄 **lib/datos/base_datos.dart**

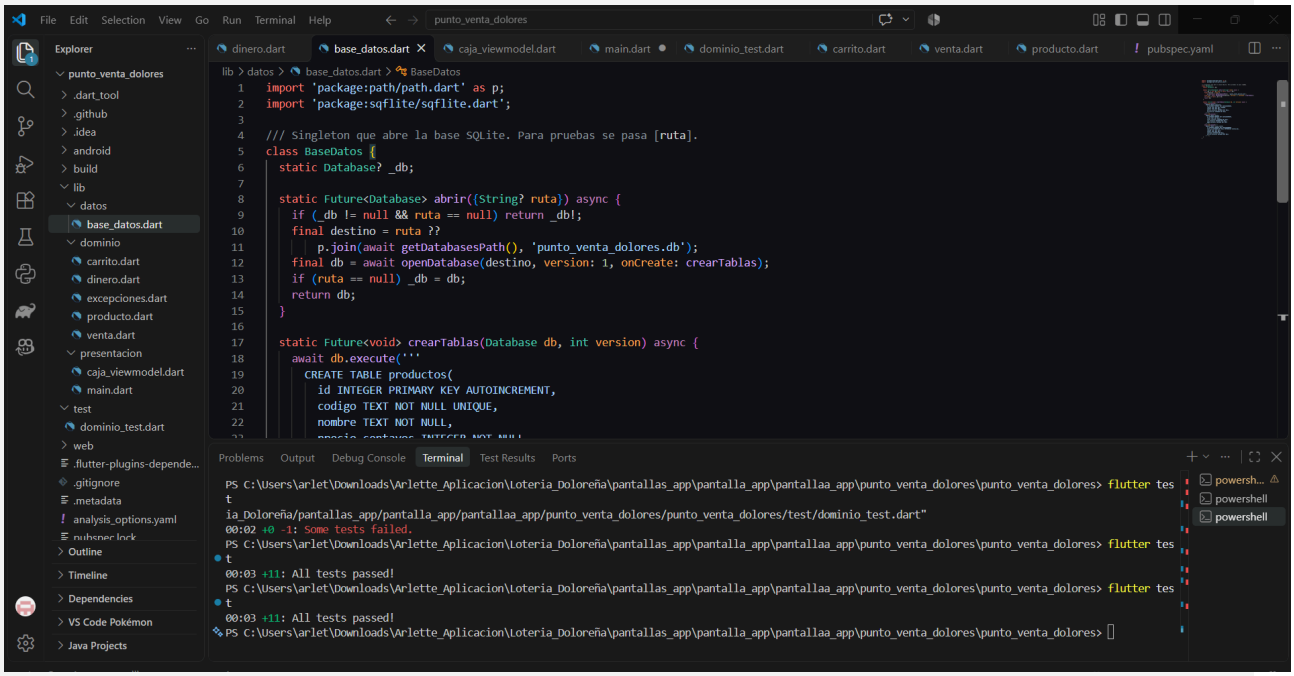
```
import 'package:path/path.dart' as p;
import 'package:sqflite/sqflite.dart';

/// Singleton que abre la base SQLite. Para pruebas se pasa [rutaMemoria].
class BaseDatos {
  static Database? _db;

  static Future<Database> abrir({String? ruta}) async {
    if (_db != null && ruta == null) return _db!;
    final destino = ruta ??
        p.join(await getDatabasesPath(), 'punto_venta_dolores.db');
    final db = await openDatabase(destino, version: 1, onCreate: crearTablas);
    if (ruta == null) _db = db;
    return db;
  }

  static Future<void> crearTablas(Database db, int version) async {
    await db.execute('''
      CREATE TABLE productos(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        codigo TEXT NOT NULL UNIQUE,
        nombre TEXT NOT NULL,
        precio_centavos INTEGER NOT NULL,
        existencias INTEGER NOT NULL
      )''');
    await db.execute('''
      CREATE TABLE ventas(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        fecha TEXT NOT NULL,
        total_centavos INTEGER NOT NULL,
        iva_centavos INTEGER NOT NULL,
        pago_centavos INTEGER NOT NULL
      )''');
    await db.execute('''
      CREATE TABLE lineas_venta(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        venta_id INTEGER NOT NULL REFERENCES ventas(id),
        codigo TEXT NOT NULL,
```

```
        nombre TEXT NOT NULL,
        cantidad INTEGER NOT NULL,
        precio_centavos INTEGER NOT NULL
      )''');
  }
}
```



> 💡 **Dato curioso / ejemplo**
> abrir({ruta}) permite elegir dónde vive la base. En producción usa un archivo; en las pruebas usamos inMemoryDatabasePath: una base en RAM que nace limpia en cada prueba y desaparece al terminar. Es como un cuaderno nuevo para cada examen.

Paso 4.2 · Configurar las pruebas de datos

📄 test/datos_test.dart (arranque)

```
import 'package:flutter_test/flutter_test.dart';
import 'package:punto_venta_dolores/datos/base_datos.dart';
import 'package:punto_venta_dolores/datos/repositorio_productos.dart';
import 'package:punto_venta_dolores/datos/repositorio_ventas.dart';
import 'package:punto_venta_dolores/dominio/carrito.dart';
import 'package:punto_venta_dolores/dominio/excepciones.dart';
import 'package:punto_venta_dolores/dominio/producto.dart';
import 'package:sqflite_common_ffi/sqflite_ffi.dart';

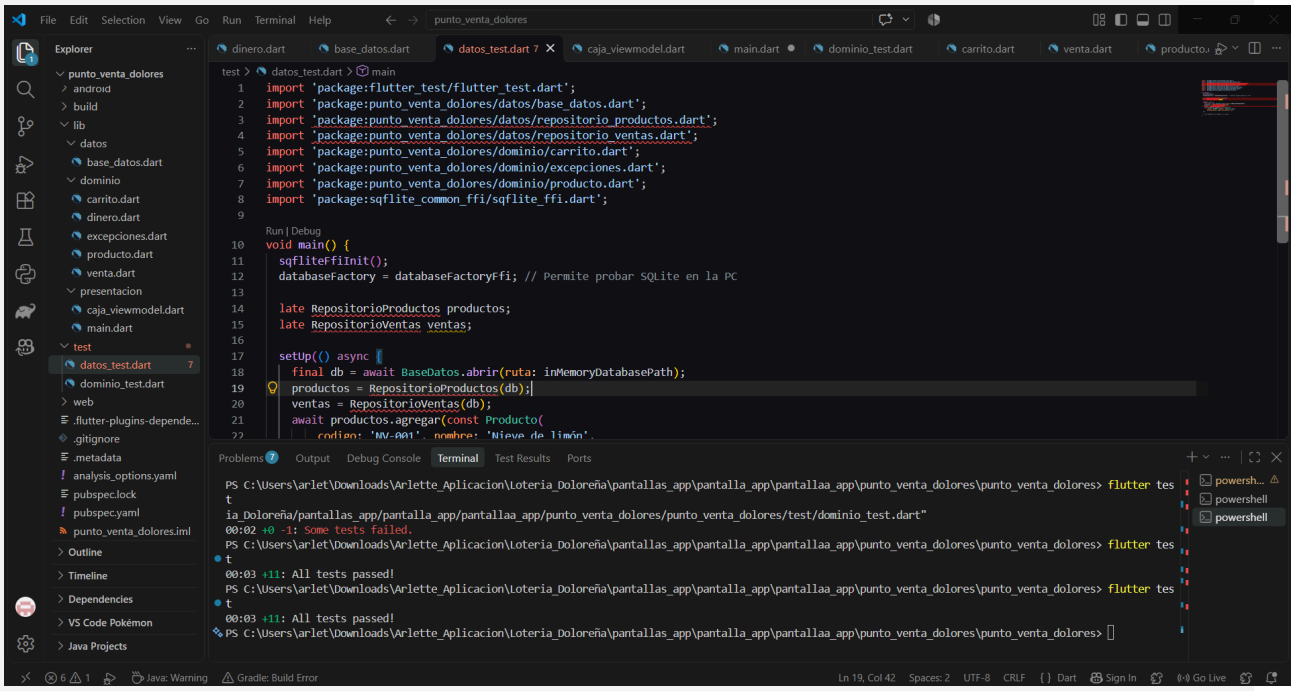
void main() {
  sqfliteFfiInit();
  databaseFactory = databaseFactoryFfi; // SQLite en tu computadora

  late RepositorioProductos productos;
  late RepositorioVentas ventas;

  setUp(() async {
    final db = await BaseDatos.abrir(ruta: inMemoryDatabasePath);
    productos = RepositorioProductos(db);
    ventas = RepositorioVentas(db);
    await productos.agregar(const Producto(
```

```
        codigo: 'NV-001', nombre: 'Nieve de limón',
        precioCentavos: 2500, existencias: 5));
  });

  // aquí van las pruebas...
}
```



> ⚠️ **Cuidado**
> setUp corre ANTES de cada prueba y crea una base nueva. Así ninguna prueba depende de otra: el orden no importa. Si una prueba contamina a otra, tendrás errores fantasma muy difíciles de encontrar.

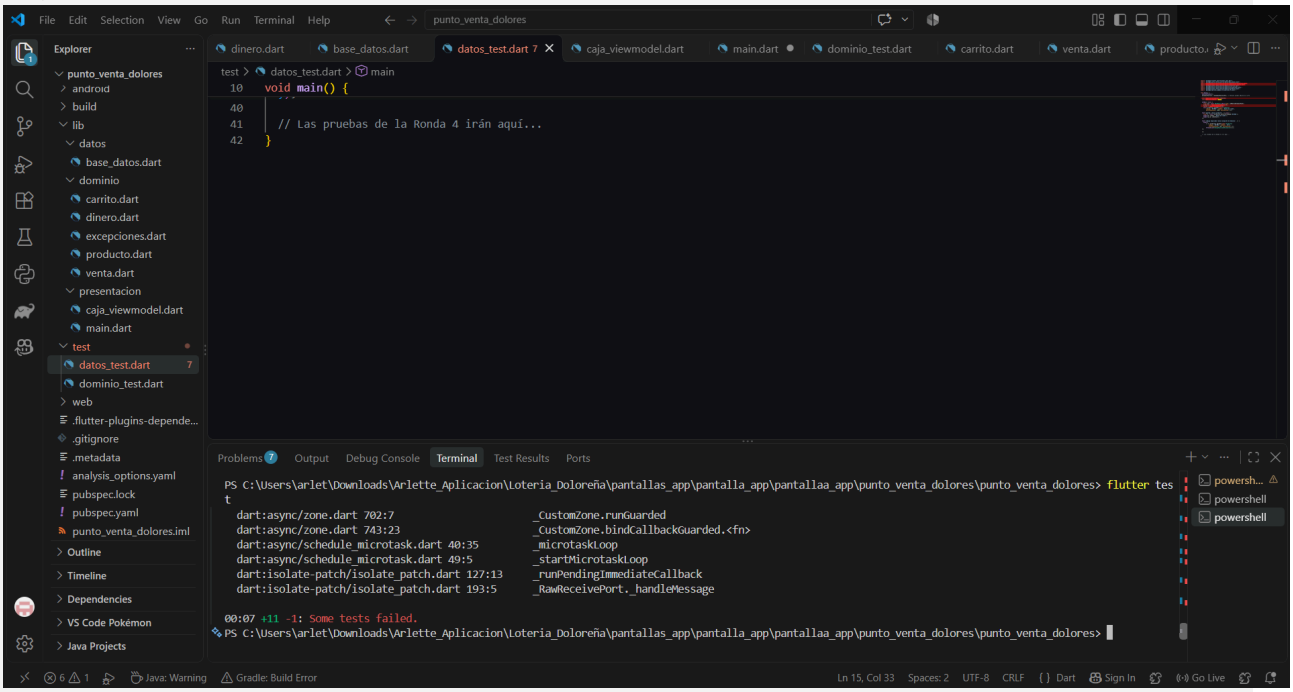
Ciclo 4.3 · Guardar y buscar

> 🔴 **ROJO: escribe la prueba (debe FALLAR)**
> Escribe dos pruebas dentro de main():

```
  test('guarda y busca producto', () async {
    final p = await productos.buscarPorCodigo('NV-001');
    expect(p!.nombre, 'Nieve de limón');
    expect(p.id, isNotNull);
  });

  test('código duplicado lanza excepción de dominio', () {
    expect(
        () => productos.agregar(const Producto(
            codigo: 'NV-001', nombre: 'Otra',
            precioCentavos: 100, existencias: 1)),
        throwsA(isA<CodigoDuplicadoException>()));
```

```
});
```



🟢 VERDE: el código mínimo para pasar

Crea el repositorio de productos:

📄 lib/datos/repositorio_productos.dart

```
import 'package:sqflite/sqflite.dart';
import '../dominio/excepciones.dart';
import '../dominio/producto.dart';

class RepositorioProductos {
  final Database db;
  RepositorioProductos(this.db);

  Future<Producto> agregar(Producto p) async {
    try {
      final id = await db.insert('productos', p.aMapa());
      return p.copiarCon(id: id);
    } on DatabaseException catch (e) {
      if (e.isUniqueConstraintError()) throw CodigoDuplicadoException(p.codigo);
      rethrow;
    }
  }

  Future<Producto?> buscarPorCodigo(String codigo) async {
    final filas =
        await db.query('productos', where: 'codigo = ?', whereArgs: [codigo]);
    return filas.isEmpty ? null : Producto.desdeMapa(filas.first);
  }

  Future<List<Producto>> listar({String filtro = ''}) async {
    final filas = await db.query(
      'productos',
      where: filtro.isEmpty ? null : 'nombre LIKE ? OR codigo LIKE ?',
      whereArgs: filtro.isEmpty ? null : ['%$filtro%', '%$filtro%'],
      orderBy: 'nombre',
    );
```

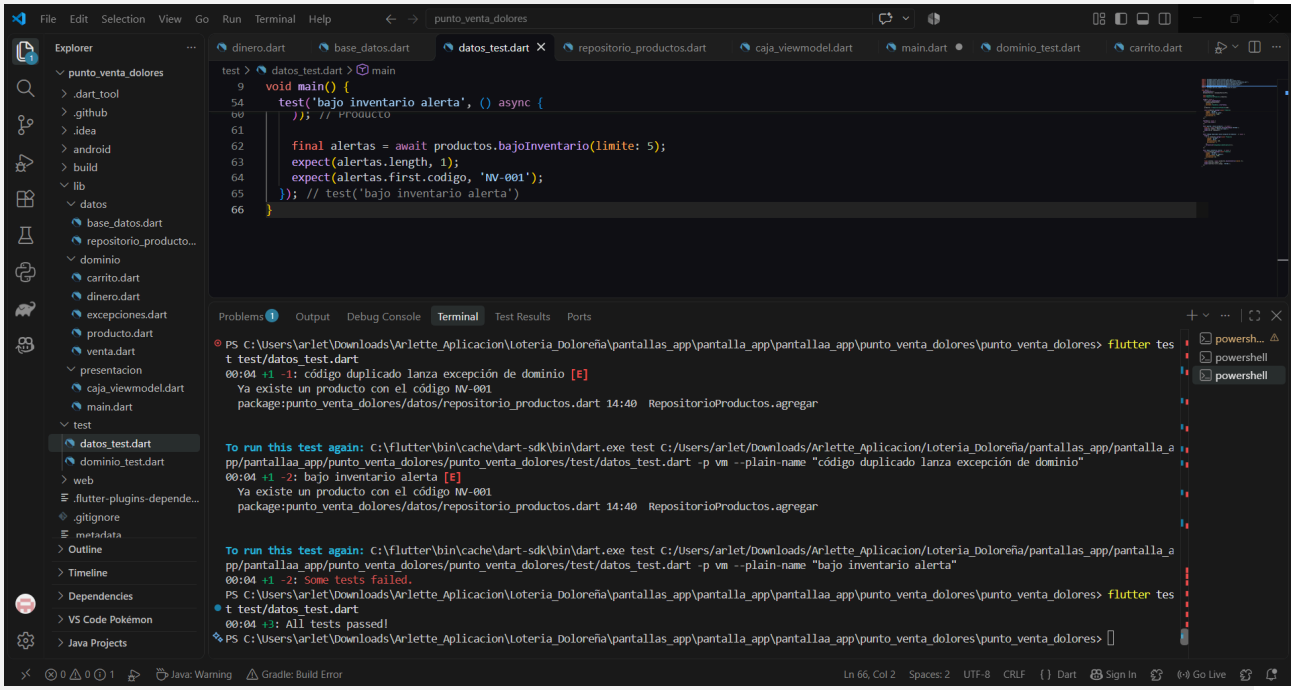
```
    return filas.map(Producto.desdeMapa).toList();
  }

  Future<void> actualizarExistencias(String codigo, int nuevas) =>
      db.update('productos', {'existencias': nuevas},
          where: 'codigo = ?', whereArgs: [codigo]);

  Future<void> eliminar(String codigo) =>
      db.delete('productos', where: 'codigo = ?', whereArgs: [codigo]);

  /// Productos con pocas existencias (alerta de resurtido).
  Future<List<Producto>> bajoInventario({int limite = 5}) async {
    final filas = await db.query('productos',
        where: 'existencias <= ?', whereArgs: [limite], orderBy: 'existencias');
    return filas.map(Producto.desdeMapa).toList();
  }

  Future<void> sembrarEjemplos() async {
    if ((await listar()).isNotEmpty) return;
    const ejemplos = [
      Producto(codigo: 'NV-001', nombre: 'Nieve de limón (1 bola)', precioCentavos: 2500,
existencias: 40),
      Producto(codigo: 'NV-002', nombre: 'Nieve de aguacate', precioCentavos: 2800,
existencias: 30),
      Producto(codigo: 'NV-003', nombre: 'Nieve de tequila', precioCentavos: 3500,
existencias: 20),
      Producto(codigo: 'DU-001', nombre: 'Dulce de leche de cajeta', precioCentavos: 1800,
existencias: 50),
      Producto(codigo: 'DU-002', nombre: 'Alegría de amaranto', precioCentavos: 1200,
existencias: 60),
      Producto(codigo: 'CE-001', nombre: 'Cerámica Talavera chica', precioCentavos: 18000,
existencias: 8),
      Producto(codigo: 'AG-001', nombre: 'Agua de horchata 1L', precioCentavos: 3000,
existencias: 25),
      Producto(codigo: 'ES-001', nombre: 'Chicharrón preparado', precioCentavos: 4500,
existencias: 15),
    ];
    for (final e in ejemplos) {
      await agregar(e);
    }
  }
}
```



💡 Dato curioso / ejemplo

El patrón Repositorio es un "almacenista": el resto de la app le pide "dame el producto NV-001" sin saber si viene de SQLite, de internet o de un archivo. Y fíjate: traducimos el error técnico de SQL (restricción UNIQUE) a nuestra CodigoDuplicadoException. El cajero nunca debe ver mensajes de SQL.

🔵 REFACTOR: ordena sin romper nada

Probamos también sembrarEjemplos(), que carga la nevería de ejemplo: nieve de limón, aguacate, tequila, alegrías, Talavera... Así la app arranca con datos.

🏆 RETO PARA TI

Agrega una prueba para bajoInventario() con un producto de 2 existencias y otro de 50. ¿Cuántos devuelve con el límite por omisión de 5?

🏅 INSIGNIA DESBLOQUEADA: Dominador de SQLite

Ronda 5 · Cobrar: la transacción

El peor escenario

Imagina esto: la app guarda la venta, pero antes de descontar el inventario se acaba la batería. Resultado: dinero registrado, pero inventario intacto; mañana venderás productos que ya no existen. O al revés: descuenta inventario pero no registra el dinero. Cualquiera de los dos es un desastre contable.

La solución: transacción

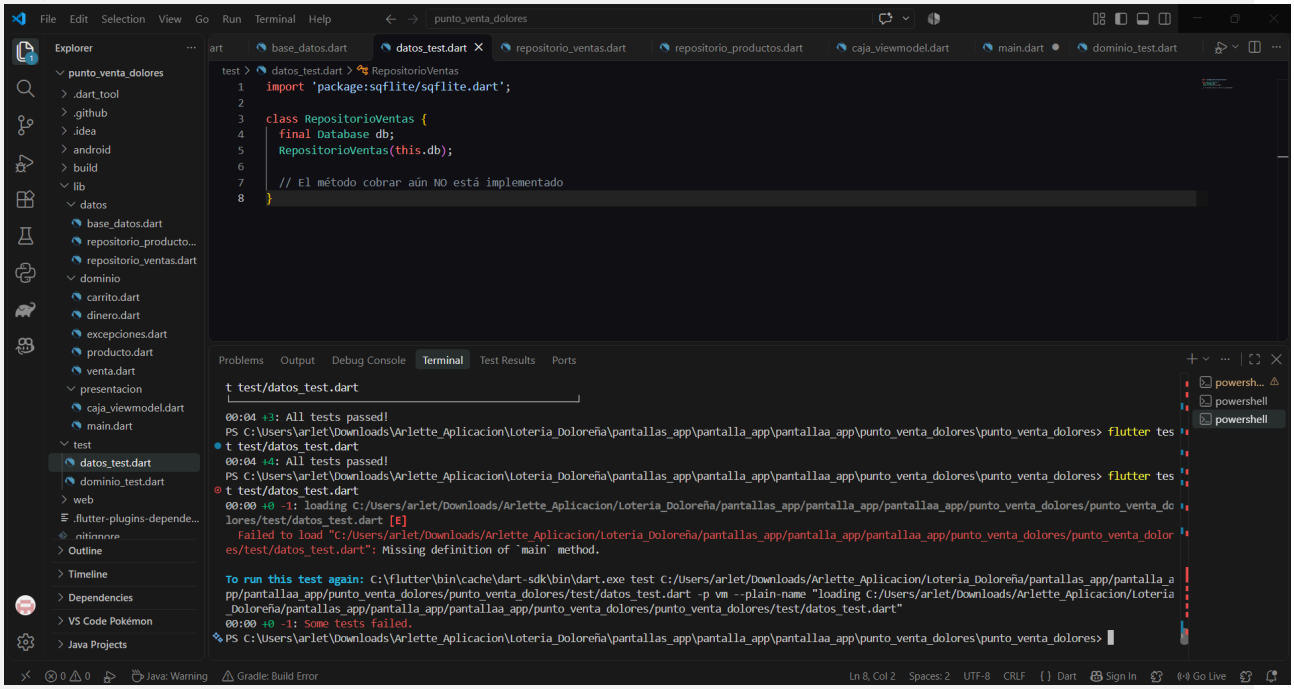
Una transacción agrupa varias operaciones en un "todo o nada": o TODAS se guardan, o NINGUNA. Es como pasar por caja: o pagas y te llevas todo, o dejas todo y te vas.

> 💡 **Dato curioso / ejemplo**
> El ejemplo clásico es la transferencia bancaria: restar $500 a Ana y sumarlos a Beto. Si se cae el sistema entre las dos operaciones, el dinero desaparecería. Las transacciones garantizan que eso no pase.

Ciclo 5.1 · Cobrar descuenta inventario

> 🔴 **ROJO: escribe la prueba (debe FALLAR)**
> Compramos 2 de 5 nieves y pagamos $100.00 (total $50.00, cambio $50.00):

```
test('cobrar descuenta inventario y registra venta', () async {
  final p = (await productos.buscarPorCodigo('NV-001'))!;
  final carrito = Carrito()..agregar(p, cantidad: 2);
  final venta = await ventas.cobrar(carrito, 10000);
  expect(venta.cambioCentavos, 5000);
  expect((await productos.buscarPorCodigo('NV-001'))!.existencias, 3);
});
```



> 🟢 **VERDE: el código mínimo para pasar**

Crea el repositorio de ventas:

lib/datos/repositorio_ventas.dart

```
import 'package:sqflite/sqflite.dart';
import '../dominio/carrito.dart';
import '../dominio/excepciones.dart';
import '../dominio/venta.dart';

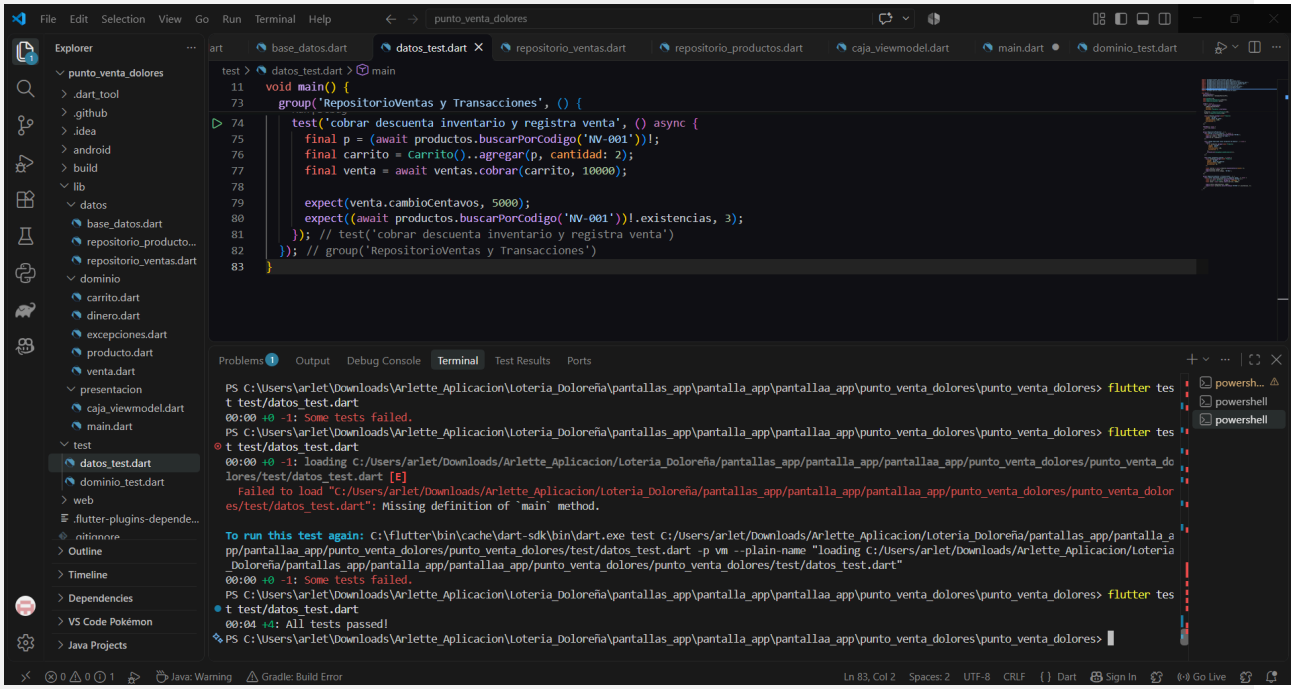
class RepositorioVentas {
  final Database db;
  RepositorioVentas(this.db);

  /// Cobra: valida pago, guarda venta + líneas y descuenta inventario.
  /// Todo en UNA transacción: o se guarda todo o no se guarda nada.
  Future<Venta> cobrar(Carrito carrito, int pagoCentavos, {DateTime? fecha}) async {
    carrito.cambio(pagoCentavos); // lanza si carrito vacío o pago corto
    final momento = fecha ?? DateTime.now();
    late int ventaId;
    await db.transaction((txn) async {
      // Revalida existencias contra la base (otro cajero pudo vender).
      for (final l in carrito.lineas) {
        final f = await txn.query('productos',
            columns: ['existencias'],
            where: 'codigo = ?',
            whereArgs: [l.producto.codigo]);
        final disp = f.isEmpty ? 0 : f.first['existencias'] as int;
        if (disp < l.cantidad) {
          throw ExistenciasInsuficientesException(l.producto.nombre, disp);
        }
      }
      ventaId = await txn.insert('ventas', {
        'fecha': momento.toIso8601String(),
        'total_centavos': carrito.totalCentavos,
        'iva_centavos': carrito.ivaCentavos,
        'pago_centavos': pagoCentavos,
      });
      for (final l in carrito.lineas) {
        await txn.insert('lineas_venta', {
          'venta_id': ventaId,
          'codigo': l.producto.codigo,
          'nombre': l.producto.nombre,
          'cantidad': l.cantidad,
          'precio_centavos': l.producto.precioCentavos,
        });
        await txn.rawUpdate(
            'UPDATE productos SET existencias = existencias - ? WHERE codigo = ?',
            [l.cantidad, l.producto.codigo]);
      }
    });
    return Venta(
      id: ventaId,
      fecha: momento,
      totalCentavos: carrito.totalCentavos,
      ivaCentavos: carrito.ivaCentavos,
      pagoCentavos: pagoCentavos,
      lineas: carrito.lineas
          .map((l) => LineaVenta(l.producto.codigo, l.producto.nombre,
              l.cantidad, l.producto.precioCentavos))
          .toList(),
    );
  }

  /// Corte del día: suma de ventas de la fecha dada.
```

```
  Future<ResumenDia> corteDelDia(DateTime dia) async {
    final inicio = DateTime(dia.year, dia.month, dia.day);
    final fin = inicio.add(const Duration(days: 1));
    final r = await db.rawQuery('''
      SELECT COUNT(*) AS n,
             COALESCE(SUM(total_centavos),0) AS total,
             COALESCE(SUM(iva_centavos),0) AS iva
      FROM ventas WHERE fecha >= ? AND fecha < ?''',
        [inicio.toIso8601String(), fin.toIso8601String()]);
    final f = r.first;
    return ResumenDia(f['n'] as int, f['total'] as int, f['iva'] as int);
  }

  /// Los más vendidos: ranking para decidir qué resurtir.
  Future<List<Map<String, Object?>>> masVendidos({int limite = 5}) =>
      db.rawQuery('''
        SELECT nombre, SUM(cantidad) AS unidades
        FROM lineas_venta GROUP BY codigo
        ORDER BY unidades DESC LIMIT ?''', [limite]);
}
```



Lectura guiada del método cobrar

1. carrito.cambio(pago) valida primero: si el pago es corto o el carrito vacío, lanza el error y NO toca la base.
2. db.transaction abre el "todo o nada".
3. Dentro, volvemos a revisar las existencias en la base: otro cajero pudo vender mientras tú llenabas el carrito.
4. Insertamos la venta y obtenemos su id.
5. Por cada renglón: insertamos la línea y descontamos con UPDATE ... existencias = existencias - ?.
6. Si algo lanza una excepción dentro del bloque, SQLite revierte todo automáticamente.

> ⚠️ **Cuidado**

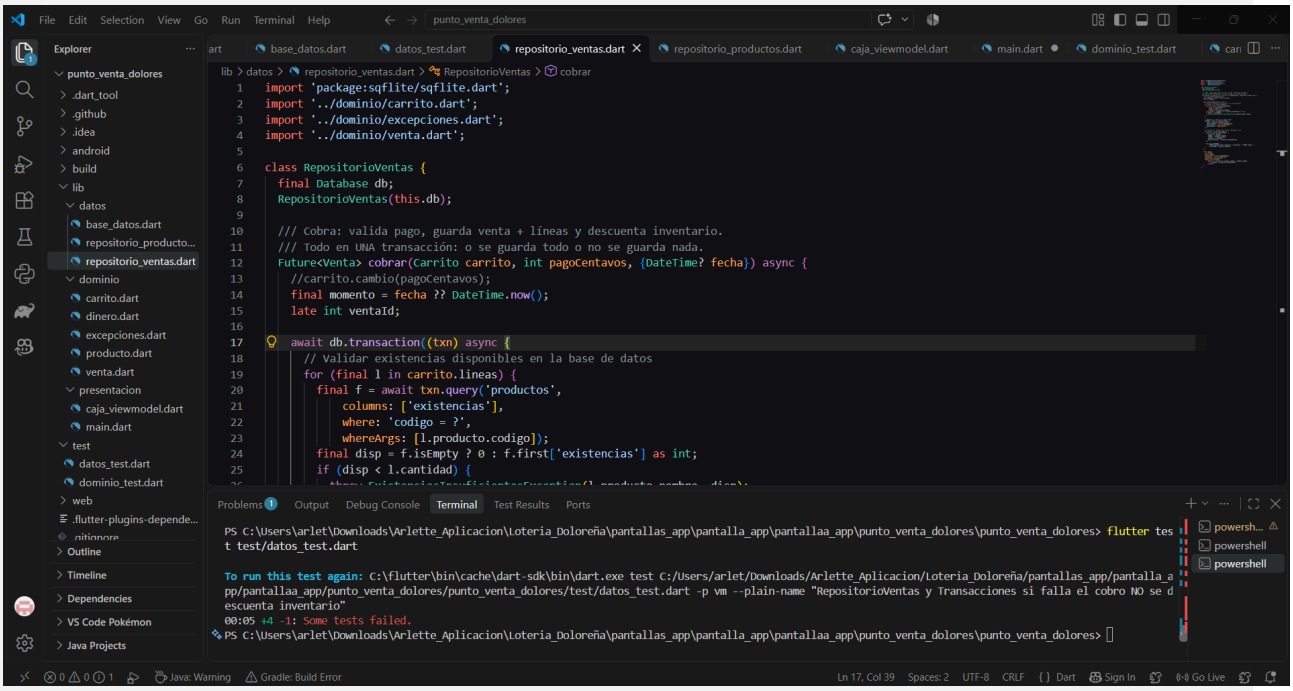
Usamos "existencias = existencias - ?" en SQL y no "leer, restar en Dart y guardar". Si dos cajeros lo hacen a la vez con la segunda forma, ambos leen 5, ambos guardan 4 y se perdió una venta de inventario.

Ciclo 5.2 · La prueba que demuestra la transacción

🔴 ROJO: escribe la prueba (debe FALLAR)

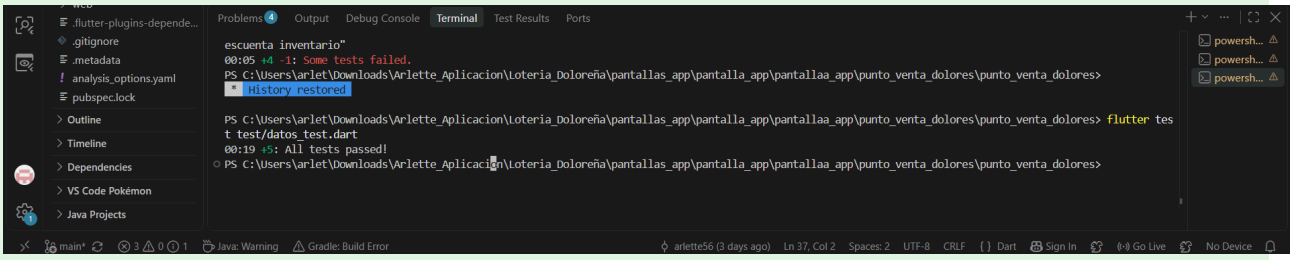
Pago insuficiente: lo importante es que el inventario NO cambie.

```
test('si falla el cobro NO se descuenta inventario', () async {
  final p = (await productos.buscarPorCodigo('NV-001'))!;
  final carrito = Carrito()..agregar(p, cantidad: 2);
  await expectLater(ventas.cobrar(carrito, 100),
      throwsA(isA<PagoInsuficienteException>()));
  expect((await productos.buscarPorCodigo('NV-001'))!.existencias, 5);
});
```



🟢 VERDE: el código mínimo para pasar

Ya pasa gracias a la validación inicial. Esta prueba es tu "seguro de vida": si algún día alguien reordena el código y rompe la regla, saltará en rojo.



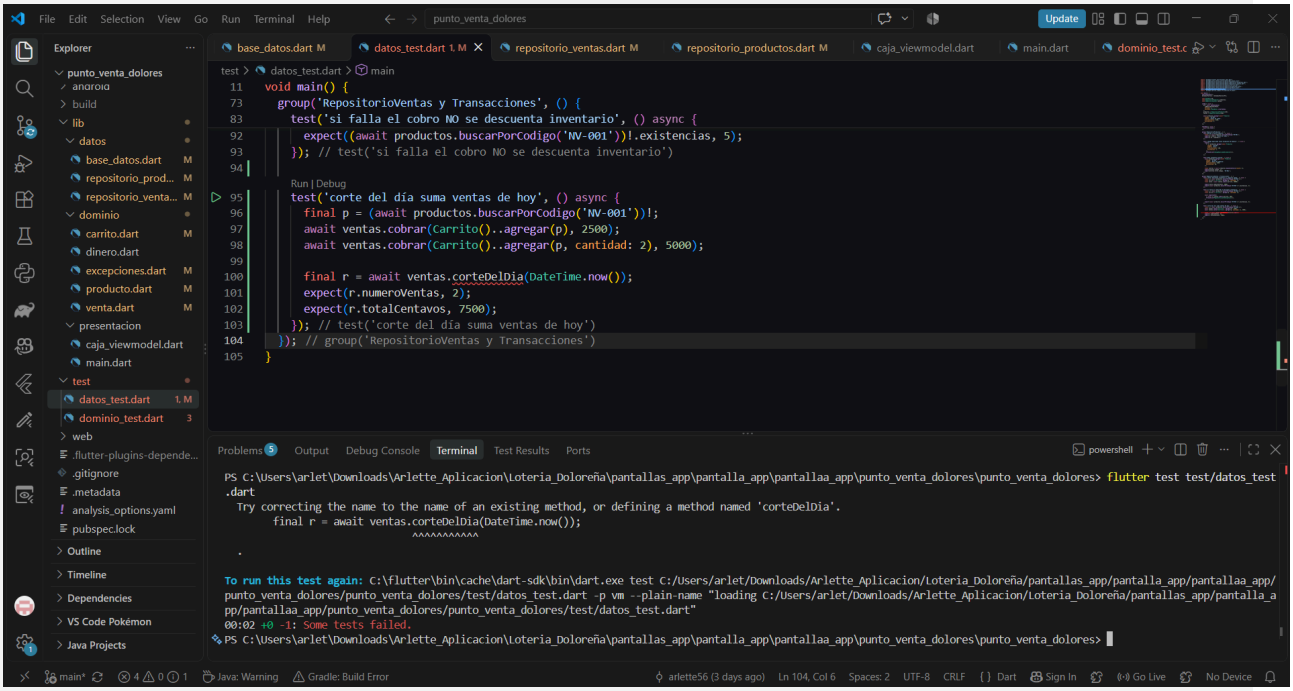
Ciclo 5.3 · Corte del día

🔴 ROJO: escribe la prueba (debe FALLAR)

Dos ventas hoy: total esperado $75.00.

```
  test('corte del día suma ventas de hoy', () async {
    final p = (await productos.buscarPorCodigo('NV-001'))!;
    await ventas.cobrar(Carrito()..agregar(p), 2500);
    await ventas.cobrar(Carrito()..agregar(p, cantidad: 2), 5000);
    final r = await ventas.corteDelDia(DateTime.now());
    expect(r.numeroVentas, 2);
    expect(r.totalCentavos, 7500);
  });

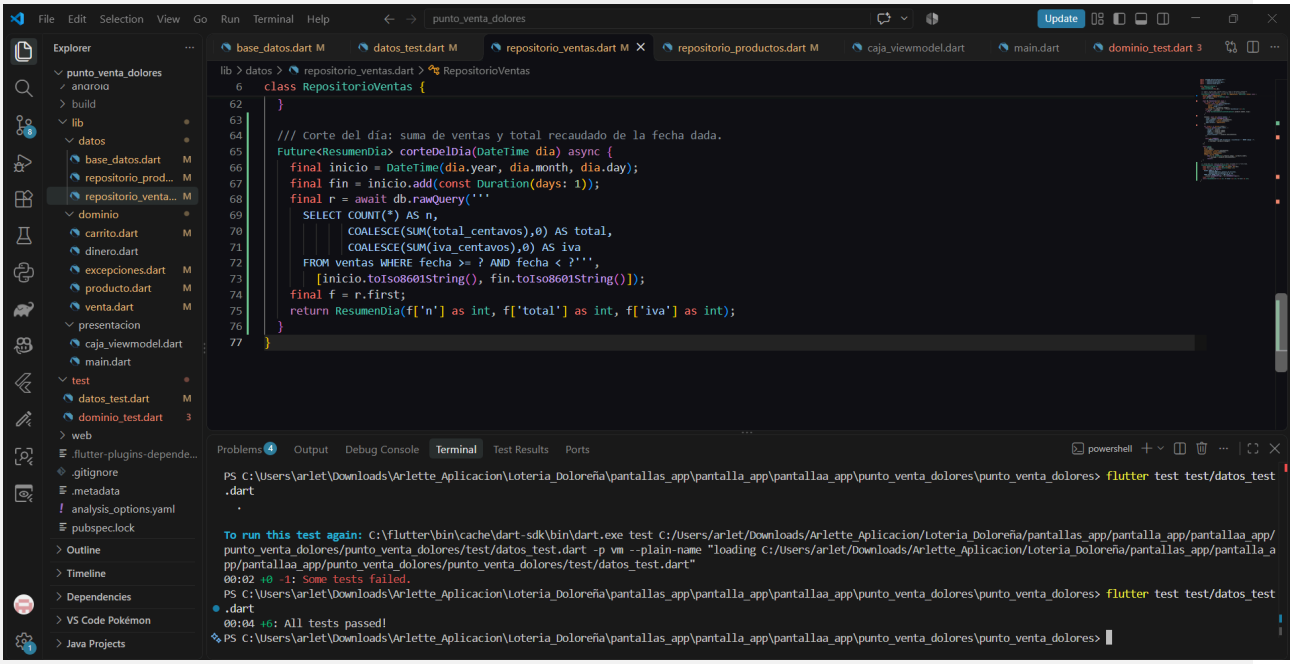
  test('bajo inventario alerta', () async {
    expect((await productos.bajoInventario()).length, 1);
  });
```



🟢 VERDE: el código mínimo para pasar

corteDelDia ya está en el repositorio: usa SUM y COUNT de SQL con un rango de fechas (de hoy 00:00 a mañana 00:00). COALESCE evita nulos cuando no hubo ventas.

```
$ flutter test
```



🔵 REFACTOR: ordena sin romper nada

Todas las pruebas deben estar en verde: dominio y datos. Ya tienes un motor de punto de venta confiable ANTES de dibujar una sola pantalla.

🏆 RETO PARA TI

Haz una prueba que venda las 5 nieves y luego intente vender 1 más. ¿Qué excepción debería lanzar cobrar()? (Pista: la revalidación dentro de la transacción).

🏅 INSIGNIA DESBLOQUEADA: Guardián de la transacción

Ronda 6 · ViewModel y pantalla

¿Por qué un ViewModel?

Si metes la lógica dentro del Widget, solo puedes probarla abriendo la pantalla. El ViewModel es un "intermediario": guarda el estado (catálogo, carrito, mensajes), llama a los repositorios y avisa a la pantalla cuando algo cambia con notifyListeners(). Así la pantalla solo dibuja.

> 💡 **Dato curioso / ejemplo**
> Es como el mesero de un restaurante: tú (la pantalla) pides, el mesero (ViewModel) lleva el pedido a la cocina (repositorios) y regresa con el resultado. Tú nunca entras a la cocina.

Paso 6.1 · El ViewModel

📄 **lib/presentacion/caja_viewmodel.dart**

```
import 'package:flutter/foundation.dart';
import '../datos/repositorio_productos.dart';
import '../datos/repositorio_ventas.dart';
import '../dominio/carrito.dart';
import '../dominio/dinero.dart';
import '../dominio/producto.dart';
import '../dominio/venta.dart';

class CajaViewModel extends ChangeNotifier {
  final RepositorioProductos productos;
  final RepositorioVentas ventas;
  Carrito carrito = Carrito();
  List<Producto> catalogo = [];
  String? mensaje;
  Venta? ultimaVenta;

  CajaViewModel(this.productos, this.ventas);

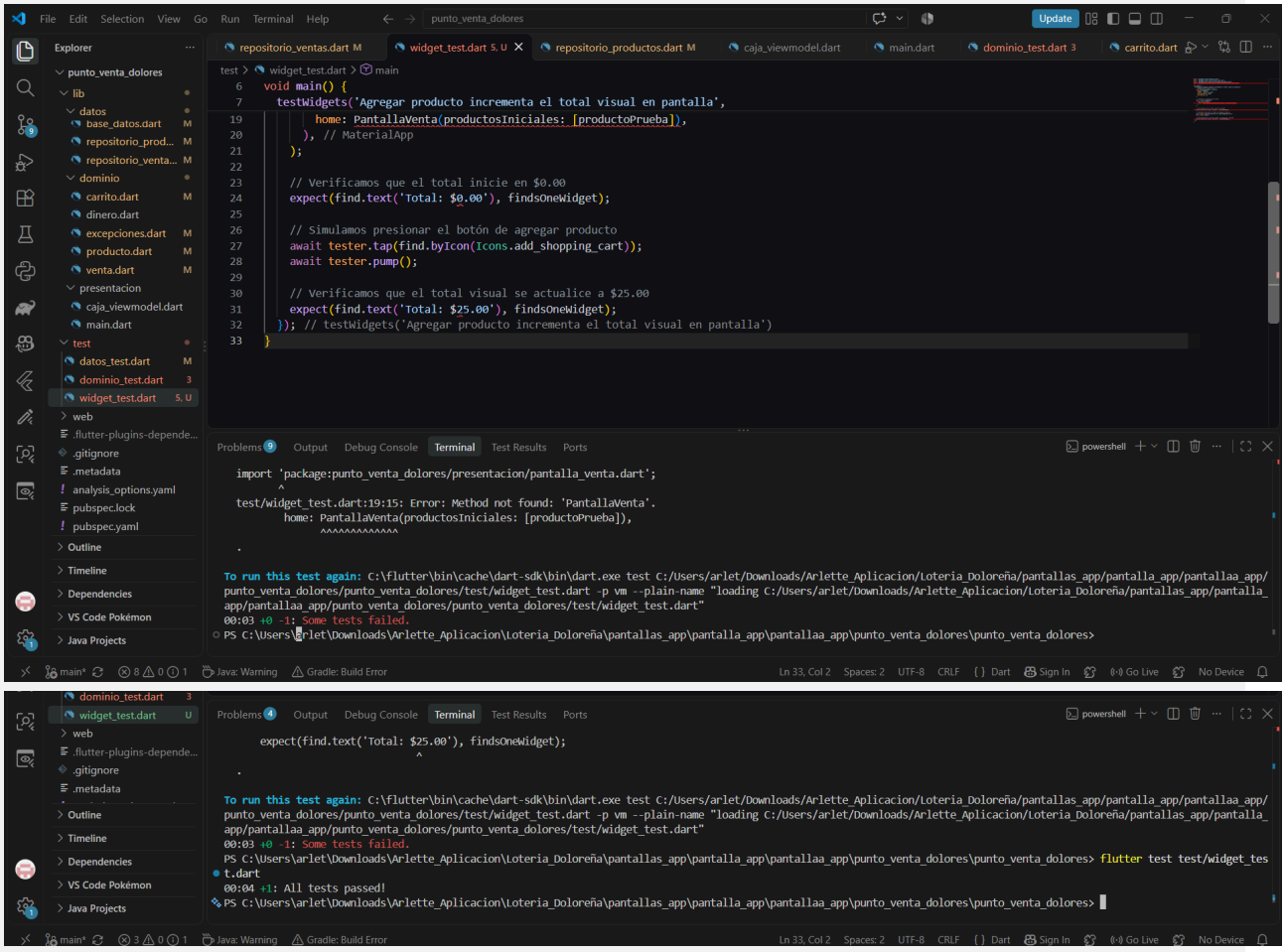
  Future<void> cargar({String filtro = ''}) async {
    catalogo = await productos.listar(filtro: filtro);
    notifyListeners();
  }

  void agregar(Producto p) {
    try {
      carrito.agregar(p);
      mensaje = null;
    } catch (e) {
      mensaje = e.toString();
    }
    notifyListeners();
  }

  void quitar(String codigo) {
    carrito.quitar(codigo);
    notifyListeners();
  }

  Future<bool> cobrar(String textoPago) async {
    try {
      final pago = pesosACentavos(textoPago);
      ultimaVenta = await ventas.cobrar(carrito, pago);
      mensaje = 'Cambio: ${formatearPesos(ultimaVenta!.cambioCentavos)}';
```

```
      carrito = Carrito();
      await cargar();
      return true;
    } catch (e) {
      mensaje = e.toString();
      notifyListeners();
      return false;
    }
  }
}
```



Puntos a observar: cobrar() recibe TEXTO (lo que escribe el cajero), lo convierte con pesosACentavos y captura cualquier excepción para mostrarla en pantalla como mensaje, sin que la app se cierre.

Paso 6.2 · La pantalla de caja

📄 lib/presentacion/pantalla_caja.dart

```
import 'package:flutter/material.dart';
import '../dominio/dinero.dart';
import 'caja_viewmodel.dart';

class PantallaCaja extends StatefulWidget {
  final CajaViewModel vm;
  const PantallaCaja({super.key, required this.vm});
  @override
  State<PantallaCaja> createState() => _PantallaCajaState();
```

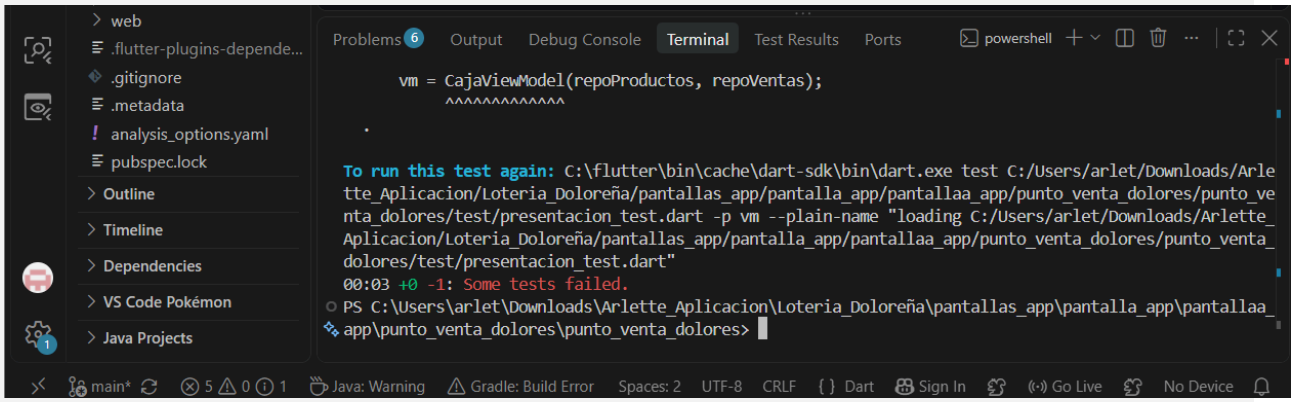
```
}

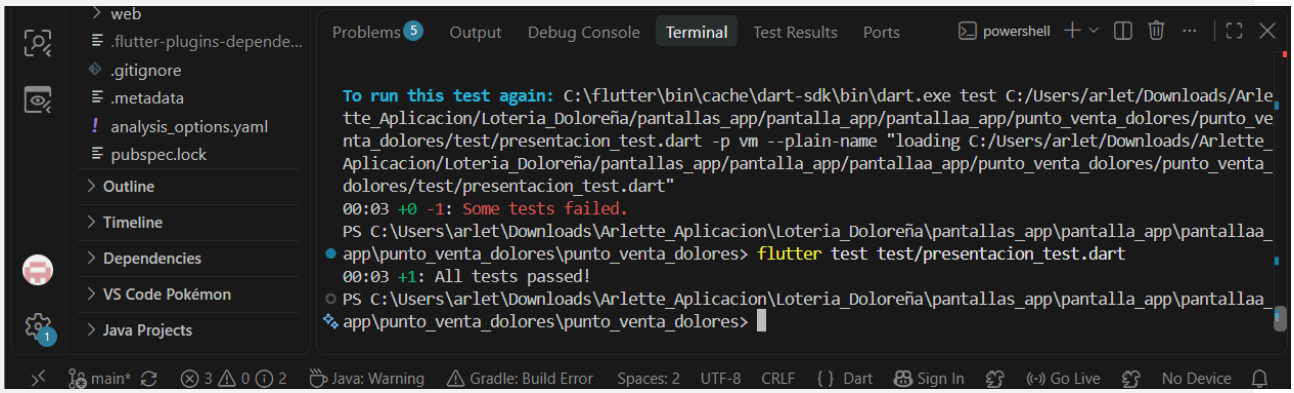
class _PantallaCajaState extends State<PantallaCaja> {
  final _pago = TextEditingController();

  @override
  void initState() {
    super.initState();
    widget.vm.cargar();
  }

  @override
  Widget build(BuildContext context) {
    final vm = widget.vm;
    return AnimatedBuilder(
      animation: vm,
      builder: (_, __) => Scaffold(
        appBar: AppBar(title: const Text('Caja · Dolores Hidalgo')),
        body: Column(children: [
          Padding(
            padding: const EdgeInsets.all(8),
            child: TextField(
              decoration: const InputDecoration(
                  prefixIcon: Icon(Icons.search), hintText: 'Buscar producto'),
              onChanged: (t) => vm.cargar(filtro: t),
            ),
          ),
          Expanded(
            child: ListView(children: [
              for (final p in vm.catalogo)
                ListTile(
                  title: Text(p.nombre),
                  subtitle: Text('${p.existencias} disponibles'),
                  trailing: Text(formatearPesos(p.precioCentavos)),
                  onTap: p.existencias == 0 ? null : () => vm.agregar(p),
                ),
            ]),
          ),
          const Divider(height: 1),
          for (final l in vm.carrito.lineas)
            ListTile(
              dense: true,
              title: Text('${l.cantidad} × ${l.producto.nombre}'),
              trailing: Row(mainAxisSize: MainAxisSize.min, children: [
                Text(formatearPesos(l.importeCentavos)),
                IconButton(
                    icon: const Icon(Icons.close),
                    onPressed: () => vm.quitar(l.producto.codigo)),
              ]),
            ),
          Padding(
            padding: const EdgeInsets.all(12),
            child: Column(children: [
              Text('IVA incluido: ${formatearPesos(vm.carrito.ivaCentavos)}'),
              Text('TOTAL ${formatearPesos(vm.carrito.totalCentavos)}',
                  style: Theme.of(context).textTheme.headlineSmall),
              Row(children: [
                Expanded(
                  child: TextField(
                    controller: _pago,
                    keyboardType:
                        const TextInputType.numberWithOptions(decimal: true),
                    decoration: const InputDecoration(labelText: 'Paga con \$'),
```

```
                ),
              ),
              const SizedBox(width: 12),
              FilledButton(
                onPressed: () async {
                  if (await vm.cobrar(_pago.text)) _pago.clear();
                },
                child: const Text('Cobrar'),
              ),
            ]),
            if (vm.mensaje != null)
              Padding(
                padding: const EdgeInsets.only(top: 8),
                child: Text(vm.mensaje!),
              ),
          ]),
        ),
      ]),
    ),
  );
 }
}
```





> 💡 **Dato curioso / ejemplo**
> AnimatedBuilder escucha al ViewModel y redibuja solo cuando llama notifyListeners(). Los productos agotados tienen onTap en null, así Flutter los deshabilita visualmente: la pantalla no puede vender lo que no existe.

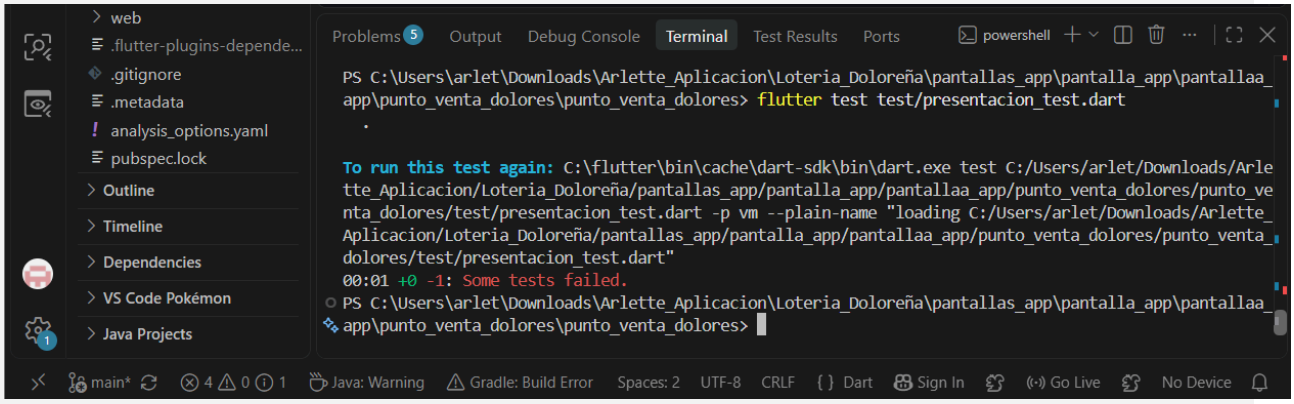
Paso 6.3 · Conectar todo en main.dart

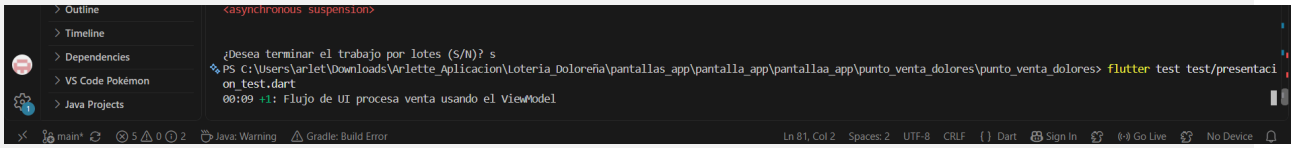
Este es el único lugar donde se crean los objetos y se conectan entre sí (composition root). El resto de la app solo recibe lo que necesita.

📄 lib/main.dart

```
import 'package:flutter/material.dart';
import 'datos/base_datos.dart';
import 'datos/repositorio_productos.dart';
import 'datos/repositorio_ventas.dart';
import 'presentacion/caja_viewmodel.dart';
import 'presentacion/pantalla_caja.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  final db = await BaseDatos.abrir(); // composition root
  final productos = RepositorioProductos(db);
  await productos.sembrarEjemplos();
  runApp(MaterialApp(
    title: 'Punto de Venta Dolores',
    theme: ThemeData(colorSchemeSeed: Colors.teal, useMaterial3: true),
    home: PantallaCaja(vm: CajaViewModel(productos, RepositorioVentas(db))),
  ));
}
```






```
$ flutter run
```

> **🔵 REFACTOR: ordena sin romper nada**
> Prueba manual: toca "Nieve de limón" dos veces, escribe 100 en "Paga con $" y toca Cobrar. Deberías ver "Cambio: $50.00" y las existencias bajar de 40 a 38.

> **⚠️ Cuidado**
> Si cierras y abres la app, los datos siguen ahí: ¡eso es SQLite! Para reiniciar de cero, desinstala la app del teléfono.

🏅 INSIGNIA DESBLOQUEADA: Constructor de interfaces

Cierre · Lo que lograste

Checklist de calidad (revisa antes de entregar)

- Todo el dinero está en centavos enteros; ningún double en el dominio.
- El dominio no importa Flutter ni sqflite.
- Cada regla de negocio nació de una prueba roja.
- Cobrar usa una transacción y se probó el caso de fallo.
- Los errores técnicos se traducen a excepciones de negocio con mensaje claro.
- La pantalla no contiene reglas de negocio.
- flutter test termina completamente en verde.

Preguntas para discutir en clase

7. ¿Qué pasaría con el corte del día si guardáramos los precios como double?
8. ¿Por qué revalidamos las existencias dentro de la transacción si el carrito ya las validó?
9. ¿Qué ventaja tiene probar con una base en memoria en lugar de la base real?
10. Si mañana cambiamos SQLite por una API en internet, ¿qué capas tendríamos que modificar?

Retos finales por equipos

- Pantalla de "Corte del día" con total, IVA, número de ventas y los 5 productos más vendidos.
- Pantalla de alta de productos con validación (precio mayor a cero, código único).
- Ticket de venta compartible como texto (usa el paquete share_plus).
- Descuentos por porcentaje y por producto, con pruebas primero.
- Adapta el sistema a otro negocio de Dolores: papelería, taquería o tienda de Talavera.

🏅 INSIGNIA DESBLOQUEADA: Cajero experto con TDD

Recuerda: el código que no tiene prueba es un deseo, no una garantía. ¡Éxito en tu punto de venta!